
A FORMAL COMPLETED PROOF OF THE NAVIER–STOKES CLOSURE FROM EXISTENCE ALONE

Global Regularity Proved from One Act on No Axiom, Every Cone Empty; Regularity of Matter Proved from the Quantum Speed Limit

FOUNDATIONS OF MATHEMATICAL ANALYSIS · UNIVERSAL CLOSURE

Nothing escapes the act

MOHAMMAD F. ISLAM, PhD

Trisduction Research Group · Independent researcher

6 October 2026

ABSTRACT. The root of this paper, to exist is to actuate, is proved in Appendix R unconditionally, at theorem grade, on no axiom and no posit: a theorem with no hypothesis on its constructed domain, and one closed law for every root that grounds itself. The closing theorem consumes one reading of that root on this row and nothing else: its premise is the root read on the row (`root_on_row_is_the_act`, `regularity_from_root_on_row`), and the lock closes at Appendix R's constructed root with no hypothesis (`the_lock_on_the_root`); every cone of the kernel is empty. The root alone decides no value, by theorem, neither forcing nor refuting it (`undeniable_root_forces_no_value`); the root read on the row is the value, by theorem (`root_on_row_is_the_value`); and the derivation of the value from the record alone is blocked, by theorem: any record shared by the regular frame and the blowup frame decides nothing (`record_wall_on_frames`), and the energy record decides nothing in either direction, a regular priced frame carrying the same energy record as the blowup one (`energy_record_decides_nothing`). A flow exists by acting, and it acts by registering: every resolved change of its state is a registration. This paper closes the regularity question of the unforced three-dimensional Navier–Stokes equations on that one reading of existence, with the freedom arrow beside it, and on nothing else. Its kernel, in core Lean 4 with no library and no axiom declared, proves sixty-five theorems, and every one depends on no axiom at all: not propositional extensionality, not quotient soundness, not choice. Existence as universally given, the root, the arrow and the bit of freedom, is proved to hold in the regular world and in the blowup world alike (`existence_as_given_in_both_worlds`); the root is proved to carry the form of the closure on every frame, the act's identity with the value, the two gates of a count, finite or unbounded, and the refuter (`given_carries_the_form`), and to add nothing to it (`root_conservative`); the arrow is given on every type and forces nothing (`arrow_given`, `arrow_forces_nothing`), and the freedom bit is given, one orbit of two with neither point its own denial (`freedom_given`). Existence read on the row, that whatever exists acts finitely at every time, is proved to be the value exactly. It is supplied by one act, as one field of one type, and global regularity follows from it by a theorem the compiler prints on no axiom. Nothing escapes the act: every instant of every datum lands on exactly one of two gates, a finite count or an unbounded one, and under the act every count is finite; one blowup is the only refuter. On matter the act is not supplied: every count is finite by theorem (`matter_acts_finitely`), since under the quantum speed limit a finite mean energy above the ground state permits finitely many registrations before any time, a singularity, under the cascade carried as a hypothesis, needs unboundedly many, and so no physical flow blows up (`no_physical_blowup`). Energy alone does not decide, and the paper proves that too. The value fibre over the record has two points, walled (`record_wall_on_frames`), and three independent axes lock it to one point; in the Bool model of the bit it is one free orbit with the shape of a prime's fibre, itself one swap orbit (`one_free_orbit`, `prime_fibre_one_swap_orbit`); the fibre of frames over the record of the first n data holds the staged world of every n , regular on its first n data and not regular (`finite_record_never_forces`), and over the energy record the value bit takes both points (`energy_record_decides_nothing`). The continuum sentence is closed on the act, at the grade of the act, and the paper states that grade in the words it proves.

KEYWORDS *Navier–Stokes equations · global regularity · existence · freedom · registration · quantum speed limit · universal closure · Lean 4*

The root of this paper is proved, on no axiom and no hypothesis, in Appendix R. The paper's closing theorem consumes one reading of that root, on this row, and nothing else, and its axiom cone is

empty. The root alone decides no value, by theorem, neither forcing nor refuting it (`undeniable_root_forces_no_value`); the root read on the row is the value, by theorem (`root_on_row_is_the_value`);

and the traditional route of derivation from the record is blocked, by theorem (record_wall_on_frames, energy_record_decides_nothing).

1. HOW TO READ THIS PAPER

From existence alone, in the title, means from existence read on the row, not from existence as given, which forces no value (from_existence_alone, nothing_given_forces). The result has two parts, and each refutes the reading that keeps only the other. The first reading notices that the act of Section 8 is equivalent to regularity and concludes that the paper assumes what it proves. The second notices a theorem whose conclusion is global regularity and concludes that the regularity problem has fallen. Both readings miss the result, and the theorem that binds the two parts compiles on no axiom: the_closure proves in one statement that existence as given forces no value and holds in both worlds; that existence read on the row is the value; that the act yields the value; that nothing escapes it; that one blowup refutes it; that energy does not decide; that no physical flow blows up; that the kinetic record, the Bool model of the bit, is walled; and that no finite record forces the value.

1.1 The verdict

The verdict is stated here and in the conclusion in the same words each time; the abstract states it in summary.

The Navier–Stokes regularity problem is closed to one act of existence. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the regular and the blowup world alike. Existence read on the row, that whatever exists acts finitely at every time, is the value, exactly. It is supplied by one act, at premise grade, as one field of one type, and the compiler prints global regularity from it on no axiom. Nothing escapes the act; one blowup is the only refuter. On matter, that no flow blows up is a theorem of the quantum speed limit in two senses: in the limit sense of blowup, stage counts before a horizon passing every bound, from the speed limit alone, on a world carrying the speed limit and nothing else (no_limit_blowup_from_speed_only) and on the physical frame (no_limit_blowup_from_speed); and in the count sense, the frame’s count by a horizon unbounded, from the speed limit with the cascade, a hypothesis

carried as a field, the limit-form link between the count and the stages, an unbounded count by a horizon having stage counts before it above every bound, not a cited physical result and not Section 2’s definition (no_actual_blowup), both fields load-bearing for the count sense (speed_is_load_bearing, cascade_is_load_bearing). Every theorem of the kernel depends on no axiom at all.

1.2 What the paper does not say

It does not say that regularity follows from existence as given: Section 3 proves that it does not, and that proof is part of the closure, not a gap in it. It does not say that the speed limit, a cited result about matter carried as a field, is a theorem about the continuum equations: the identification of the two is the reader’s, and Section 11 types it. It does not engage the correctness of the forced construction released in September 2026: Section 13 shields the unforced row from it.

1.3 The reflexive readings, and the theorem that answers each

1.4 The lock

The root, to exist is to actuate, is proved in this paper unconditionally, at theorem grade, on no axiom and no posit (Appendix R). On its constructed domain it is a theorem with no hypothesis (root_on_the_constructed_domain; in the row kernel, constructed_root_holds), and it is satisfiable on every background (root_satisfiable); for every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds to it (the_floor_is_universal); and one theorem binds the root’s universal law and the constructed root with the Return and the twelve gates, the gates of the quaternions in Appendix R and not the two gates of a count, counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4 (the_master_seal); the seal states the three counts side by side. The row kernel carries the root’s structure in the words of Appendix R (SelfGrounding), whose acts this paper calls deeds, keeping *the act*, unqualified, for existence read on the row: a denial of the root is a deed and instances it (denial_reenacts_root), and no outside proof adds anything to it (external_proof_adds_nothing). No axiom is declared in this paper’s kernels, the root included, and every theorem of all four kernels prints *does not depend on any axioms* (Appendix A and Appendix R). The root holds in the world where the row’s value holds and in the world where it fails, so the root alone forces no value and refutes none, the conjuncts of one theorem: the root, the value in the one world, its failure in the other, no forcing and no refuting (undeniable_root_forces_no_value; given_is_not_the_value). Read on the row, with the row’s exis-

Reading	What the kernel proves	Theorem (cone)
It is only a premise.	The act is the root's own formula read on the row. The root's undeniability stays with the root and does not pass to that reading, which grounds itself exactly where the value holds; that is why the act is keyed and decides what the root alone cannot.	<code>root_on_row_is_the_act,</code> <code>undeniability_stays_with_the_root,</code> <code>row_root_grounds_iff_value,</code> <code>root_on_row_is_keyed</code> (none)
The act is the conclusion, so the proof is circular.	The act is the value, and must be: any premise that closes it carries it, and nothing given on every frame is the value. The isolation is the result.	<code>act_is_the_value,</code> <code>act_is_the_weakest_premise,</code> <code>given_is_not_the_value</code> (none)
Existence is universally given, so the value should follow.	Existence as given holds in the regular and the blowup world; what follows from it uniformly holds without it.	<code>existence_as_given_in_both_worlds,</code> <code>root_conservative</code> (none)
Energy is finite, so nothing blows up.	The energy inequality holds on a frame where a datum blows up.	<code>energy_does_not_decide</code> (none)
Actual fluids never blow up.	Correct: no physical flow blows up, proved under the speed limit with the cascade, a hypothesis carried as a field. The continuum sentence stands on the act.	<code>no_physical_blowup,</code> <code>speed_is_load_bearing,</code> <code>cascade_is_load_bearing</code> (none)
Enough data will settle it.	The first n data are regular and the value fails, for every n .	<code>finite_record_never_forces</code> (none)
Scale small data up.	No symmetry carries a small datum onto a large one; a uniform step reaches every size.	<code>no_symmetry_lifts,</code> <code>uniform_step_forces_all</code> (none)
It can simply be rejected.	Every instant lands on a finite count or an unbounded one, never both; one unbounded count refutes the act.	<code>every_instant_lands,</code> <code>gates_exclusive,</code> <code>blowup_refutes</code> (none)

Table 1 | The reflexive readings. Every cone is empty.

tents and the row's actuation, the root is the act (`root_on_row_is_the_act`), keyed where the root alone is not (`root_read_on_row_is_keyed`, `act_is_keyed`), and the act is the row's value exactly, the two unfolding to one formula (`act_is_the_value`). The closing theorem consumes that one reading and nothing else (`regularity_from_existence`); no reading of the kinetic record returns the bit (`record_wall`), and no finite record forces it (`finite_record_never_forces`). One theorem on no axiom binds the constructed root; the root held by its deed; the refusal of every given statement; the root read uniformly, which gives no value; the root read on the row, equal to the act; the act, equal to the value; both keyed; the closing theorem; the record wall; and the finite record, for every self-grounding root (`the_lock`). Instantiated at the constructed root of Appendix R it is closed, with no hypothesis: that root holds, forces no value on every frame, read on the row is the value exactly, and the closing theorem gives the value on every actual frame (`the_lock_on_the_root`); the row kernel repeats Appendix R's ΔE_0 , RA_0 and ra_0 verbatim, and that root is the constructed root (`RA_0_is_the_constructed_root`); and the closing theorem's premise, `ActsFinitely`, is the root read on the row (`root_on_row_is_the_act`), so the value follows from that reading (`regularity_from_root_on_row`). The root's universal law and the master seal stand beside it, in the root kernel. *From existence alone* carries exactly this sense: the closing theorem consumes existence read on the row and no other premise, axiom or cited theorem; that reading is available on exactly the frames where the value holds; and no statement given without it is the value on every frame (`from_existence_alone`).

Status

Unconditional, at theorem grade, on no axiom. The root, on its constructed domain, satisfiable on every background, and for every root that grounds itself (Appendix R, `root_`

`satisfiable`); the root read on the row, equal to the act and to the value, and keyed (`root_on_row_is_the_act`, `root_on_row_is_the_value`, `root_read_on_row_is_keyed`); the closure from the reading; the record wall, in the Bool model and on frames (`record_wall`, `record_wall_on_frames`); the lock, closed at the constructed root with no hypothesis (`the_lock_on_the_root`) and for every self-grounding root (`the_lock`); every theorem of the kernel, the kernel's sixty-five, every cone empty, and every theorem of the three kernels of Appendix R, Appendix R's sixty-five in three kernels, every cone empty.

On the reading, at premise grade. Global regularity on the frame: whatever exists acts finitely at every time; read through the identification of Section 2, every strong solution of the unforced equations stays regular.

Everything except the bit is proved with no posit and no axiom: the root, the identity of its reading on the row with the act and the value, the wall and the closure, every cone empty; and the bit, the reading supplied on the actual frame, is all that remains of the row's value, at premise grade; the matter theorem of Section 11 carries the cited speed limit and the cascade as fields of its own input type, at their own grades.

2. THE CLAIM, STATED WHOLE

Let u solve the unforced incompressible Navier–Stokes equations, in the setting of the regularity problem (Feferman 2006), whose smooth rapidly decaying data lie inside the finite-energy class used here,

$$\begin{aligned}\partial_t u + (u \cdot \nabla)u - \nu \Delta u + \nabla p &= 0, \\ \nabla \cdot u &= 0, \quad u(\cdot, 0) = u_0,\end{aligned}$$

on $\mathbb{R}^3$ with viscosity $\nu > 0$, from a smooth divergence-free datum u_0 of finite energy. Read the flow as an existent: it exists by acting, and it acts by registering resolved changes of its state. Write $N(u_0, t) \in \mathbb{N} \cup \{\infty\}$ for the registrations of the flow from u_0 by time t ; the registration counted by N is an event; the registration of the statement of the Root Axiom's nature and of Ap-

pendix R.2 is the map *reg*, the fold’s record. The flow is *regular* when

$$\forall t \exists n \in \mathbb{N} : N(u_0, t) = n,$$

and it *blows up by time* t when $N(u_0, t) = \infty$. The kernel works on exactly this skeleton: a frame of data, each with a count at each time, none where the count is unbounded. That an analytic blowup of a strong solution, an unbounded critical norm at a finite time, is an unbounded count is the reader’s identification, declared here and in Definition 9.1; the kernel proves nothing about it.

The paper does three things and keeps them apart. It proves, on no axiom, everything the skeleton admits: what existence as given carries and does not carry, what the record decides and does not, what energy and the speed limit price, and the closure from one act. It isolates the one object the skeleton cannot supply, the act at the continuum, and proves it equal to the value. And it supplies that act once, in the open, as the field supply of the structure *ActualFlows*, from which global regularity follows by a theorem whose cone is empty.

2.1 What is new

The template is the author’s Riemann closure and its master volume (Islam 2026g, 2026h), and the physical reading of registration is that of the author’s operating system (Islam 2026i). The author’s earlier papers proved a continuation criterion on the alignment defect of the vorticity and named an effective premise to close it (Islam 2026a, 2026b), closed four reading routes to that premise (Islam 2026e), shielded the unforced row from forced constructions (Islam 2026c, 2026d), and built a kinetic witness on Burgers’ equation (Islam 2026f). The effective premise was a placeholder written before the formal register and the physical operating system existed. This paper retires it and reduces the row to existence alone. It adds the proof that existence as given holds in both worlds and forces no value; the act as existence read on the row, equal to the value; the universal closure with its exclusive gates, proved without excluded middle; the kinetic theorem from the quantum speed limit with the cascade, a hypothesis carried as a field that links the count to the stages in limit form, which proves regularity on matter rather than observing it; the proof that energy does not decide; the freedom of the row, with the shape of a prime in the Bool model of the bit, and the triaxial lock; and a kernel in which every theorem depends on no axiom.

3. EXISTENCE PLACED: WHAT IS GIVEN, AND WHAT IT CARRIES

Existence is given universally, and the paper takes that seriously enough to prove exactly what universality carries. The root in its formal reading, that to exist is

to actuate,

$$\text{Root}(U, \Delta E) : \iff \forall x \in U : 0 < \Delta E(x),$$

is satisfiable on every background (*root_satisfiable*), and holds vacuously on the empty one (*root_given*). The arrow, a map on every type, holds everywhere (*arrow_given*). The freedom bit, one orbit of two with neither point its own denial, holds everywhere (*freedom_given*).

What holds everywhere holds in both worlds: the arrow holds in the regular world and in the blowup world (*existence_as_given_in_both_worlds*). Under any root, on every frame, the act is the value, every instant lands on a gate and one blowup refutes the act: existence as given carries the form of the closure (*given_carries_the_form*). Whatever follows from the root uniformly in its symbols holds without it (*root_conservative*). No statement that reads the same on every frame is the value (*given_is_not_the_value*), and in particular the arrow forces nothing (*arrow_forces_nothing*). This is the first theorem of the closure, not a limit of it. Existence as given is the ground on which every frame stands, and a ground on which both worlds stand cannot be the difference between them. The difference is the value, and Section 8 locates it.

The root is more than given. It is universally presupposed and undeniable in deed: whoever examines it, doubts it or denies it acts, and every denial of a self-grounding root is a deed that instances it (*denial_reenacts_root*); no outside proof adds anything to it (*external_proof_adds_nothing*). It holds on every frame once any deed occurs, and every examination of it is one; in its formal sense, a self-grounding root indexed by frame holds on every frame and forces the value on no frame uniformly: no derivation of the value on every frame from the family exists (*grounded_on_every_frame_forces_no_value*). It is held by its deed in the world where the value holds and in the world where it fails alike, and for that very universality it fixes the form of every closure and decides no value by itself, neither forcing the value nor refuting it, both conjuncts of one theorem (*undeniable_root_forces_no_value*). Its undeniability stays with the root: the constructed root is grounded, it does not carry over to its reading on every frame, and the row’s reading is not grounded on every frame, not such a family, since it grounds itself exactly where the value holds (*undeniability_stays_with_the_root*, *row_root_grounds_iff_value*). Read on the row it becomes the act, the root’s own formula with the row’s existents and the row’s actuation (*root_on_row_is_the_act*), keyed where the root is not (*root_read_on_row_is_keyed*): the act stands at premise grade, the one premise of the row, and the root beneath it is proved at theorem grade (Appendix R).

4. THE ROUTE LEDGER

Route	Reach	Theorem
Existence as given	forces no value; holds in both worlds	given_is_not_the_value, existence_as_given_in_both_worlds
Energy	does not decide: the inequality survives a blowup	energy_does_not_decide
A finite record	does not decide	finite_record_never_forces
Symmetry	does not lift small data	no_symmetry_lifts
A uniform step in the size	reaches every size on a ladder that carries it	uniform_step_forces_all
The speed limit and the cascade, on matter	decide: no physical flow blows up, the frame of matter is regular, and both fields are load-bearing	no_physical_blowup, matter_regular_from_speed, speed_is_load_bearing, cascade_is_load_bearing
Existence read on the row	is the value, by one act	act_is_the_value, regularity_from_existence

Table 2 | *The route ledger. Every cone is empty.*

Every route to the value is a method, and each method's reach is a theorem of the kernel.

The ledger closes on its last two rows. On matter the value is reached by a theorem; at the continuum it is reached by the act. No other row reaches it. The last two reach it from named fields: matter from speed and cascade (speed, cascade), the continuum from supply (supply). The first three force no value, by the theorems they cite; symmetry lifts no small datum to a large one (no_symmetry_lifts); and the uniform step reaches every size only on a ladder that carries one (uniform_step_forces_all).

5. THE FRAME, AND THE ONE CUT

A frame registers each datum at each time. The record of the row, what observation and the price of dissipation report, reads the same in the regular world and in the blowup world, and any record that reads the two alike decides nothing: no reading of it is the value on every frame (record_wall_on_frames). The paper uses four records, each under its own word: the kinetic record, the Bool model of the bit, whose wall is record_wall; the regularity record of the first n data, which never forces the value (finite_record_never_forces); the energy record, energy and dissipation paid, which a regular priced frame shares with the blowup priced frame, so that it decides nothing in either direction (energy_record_decides_nothing); and the fold's record, the word of the statement of the Root Axiom's nature, the image of the registration of an actuation, which keeps the height, forgets the side, reads a point and its reversal alike and lands on the seat (Appendix R.2, registration_is_the_folds_record, reg_lands_on_seat). Each record of the row is scoped to its own theorem: the kinetic record keeps everything the two worlds share and forgets which world is actual, in the Bool model (record_wall); the energy record reads the regular priced frame and the blowup priced frame alike, so no reading of it is the value on every frame (energy_record_decides_nothing, record_wall_on_frames); and the regularity record of the first n data is read on the staged world alone, regular on its first n data with the value failing, so that record forces no value (finite_record_never_forces). The word cut carries two senses: in the statement of the Root Ax-

iom's nature the cut is the triaxial cut, an involution with its registration and its seat; in Appendix R.2 the only special cut is the seat, where both blind spots cancel (the_only_special_cut, seat_cancels_both). On this row the value bit's swap of the two worlds over one record, the Bool model (record_wall, fibre_is_two, one_free_orbit), has no fixed point (freedom_given); the triaxial cut is carried by the root's actuation, whose seat is the record point a point and its reversal both land on (Appendix R.2, open_at_the_seat), and the one cut of this section's title is that cut, this paper's kernel stating no seat of its own. Sections 6 and 7 read their theorems, structurally, as statements about what the record of the row keeps and what it forgets.

6. FREEDOM: THE TWO WORLDS, AND THE PRIME'S SHAPE

Over the record the value fibre has two points, the world where the value holds and the world where it fails, and no reading of a record the two worlds share is the value on every frame (record_wall_on_frames). In the Bool model of the bit, the kinetic record, the fibre has exactly two points (fibre_is_two), neither its own denial (freedom_given); negation moves every world and every world is the given one or its negation, one free orbit (one_free_orbit); and no reading of that record returns the world (record_wall). Two is the count of the value bit, not of the frames over a record: the regular priced frame and the blowup priced frame carry one energy record, energy one and dissipation one at every datum, and differ in the value, so the value bit over that energy record takes both of its points (energy_record_decides_nothing); and the fibre of frames over the record of the first n data holds the staged world of every n , regular on its first n data and not regular (finite_record_never_forces), and with it the staged world of every $m \geq n$, regular on its first m data and so on its first n , infinitely many frames over one record. This is the freedom of the row, and in the Bool model it has the shape of a prime; the comparison is structural, the kernel proving the two counts and the orbit structure and no map between the fibres. The multiplicative fibre over a prime p ,

$$\{(a, b) \in \mathbb{N}^2 : ab = p\} = \{(1, p), (p, 1)\},$$

is exactly two points, off the seat $a = b$; the kernel checks at $p = 7$ that the two counts agree (prime_shape) and that swapping the factors of each point reverses the fibre, one swap orbit (prime_fibre_one_swap_orbit). One free orbit carries one bit. The record leaves exactly that bit, as a prime leaves exactly one.

7. THE TRIAXIAL LOCK

Over $\mathbb{F}_2^3$, two independent linear axes leave exactly two points for every target, and three independent axes lock exactly one:

$$\begin{aligned} \#\{x \in \mathbb{F}_2^3 : r_1 \cdot x = t_1, r_2 \cdot x = t_2\} &= 2, \\ \#\{x \in \mathbb{F}_2^3 : r_i \cdot x = t_i, i = 1, 2, 3\} &= 1, \end{aligned}$$

for independent r_i and every target (two_axes_leave_two, three_axes_lock_one). The two axes of the record, observation and price, are the kinetic line; its two points are the two worlds of Section 6; the act of Section 8 is the third axis, and with it the lock forms on one point. This reading of $\mathbb{F}_2^3$ is the paper's structural reading in its own coordinates, observation, price and the act, three independent axes; it is a second statement beside the actuation's triple of Appendix R.2, the fold, the registration and the orientation, and the paper claims no map between the two. The counts are theorems; their reading onto the row is structural.

8. THE ACT: EXISTENCE READ ON THE ROW

Existence read on the row is the statement that whatever exists acts finitely at every time,

$$\begin{aligned} \text{ActsFinitely}(F) \\ : \iff \forall d \forall t \exists n : N_F(d, t) = n. \end{aligned}$$

The kernel proves it is the value, exactly (act_is_the_value), and that it is keyed: it holds in the regular world and fails in the blowup world (act_is_keyed). Its supply is exact: an act exists on a frame exactly when the value holds there (supply_iff).

The equivalence is the strength of the closure, as the equivalence of least erasure with the critical line is the strength of the author's Riemann closure (Islam 2026g). Any premise that closes the value implies the act, and the weakest such premise is the act, the value itself (act_is_the_weakest_premise, from act_is_the_value); and Section 3 proved that nothing given on every frame is the value. A closure that assumed less would be wrong, since every premise that forces the value implies the act (act_is_the_weakest_premise); a closure that assumed more would close the question from a premise the question does not need. The act assumes exactly the value, read as an act of existence, and nothing beside it in the kernel; the reading of the frame as the equations is the reader's identification, declared in Section 2.

9. THE PROOF: GLOBAL REGULARITY FROM EXISTENCE

Definition 9.1 (ActualFlows). A structure with two fields: F , the frame, standing for the strong solutions of the unforced equations with their registrations, the identification, blowup with an unbounded count included, being the reader's; and supply, existence acting finitely on that frame. The second field is the assumption of the closing theorem, and its only one; the matter theorem of Section 11 carries its own fields, the cited speed limit and the cascade, a hypothesis carried as a field, in its own input type.

Theorem 9.2 (regularity_from_existence). For every A : ActualFlows, every datum of $A.F$ is regular. *Cone: none.*

The proof of global regularity is Theorem 9.2. Its only assumption is one field of its input type, visible in the statement; its cone is empty, without even the standard axioms; and outside this section the structure enters only the theorems that read its supply or carry the closure whole (supply_iff, nothing_escapes, the_closure, the_lock, the_lock_on_the_root, from_existence_alone).

10. NOTHING ESCAPES

Every instant of every datum lands on exactly one gate, a finite count or an unbounded one (every_instant_lands, gates_exclusive), with no third landing. The dichotomy is proved without excluded middle, because a count is a value and not a proposition. Under the act every datum at every time acts finitely (nothing_escapes). One unbounded count refutes the act (blowup_refutes), and wherever the act fails it cannot be that no datum blows up (only_refuter). The closure is universal in the exact sense the gates give it: there is no datum, no time and no instant that the act does not decide, and there is one form, a blowup, in which it could be refuted.

11. THE PRICE: WHAT ENERGY DOES NOT DECIDE, AND WHAT THE SPEED LIMIT DOES

Energy does not decide. The energy inequality,

$$\begin{aligned} \frac{1}{2} \|u(t)\|_{L^2}^2 + \nu \int_0^t \|\nabla u(s)\|_{L^2}^2 ds \\ \leq \frac{1}{2} \|u_0\|_{L^2}^2, \end{aligned}$$

bounds the dissipation paid by the energy (Leray 1934), and it holds on a frame where a datum blows up (energy_does_not_decide). This is the frame-level face of supercriticality, whose analytic form is that the energy scales as λ^{-1} under u_λ : a finite energy budget is compatible with a singularity, because the singularity concentrates at scales where the energy cost vanishes.

The speed limit decides, on matter. A physical flow is a quantum system, and the quantum speed limit (Mandelstam and Tamm 1945; Margolus and Levitin 1998) bounds the time to reach an orthogonal state by $\pi\hbar/(2E)$, with E the mean energy above the ground

state (Margolus and Levitin) or the energy spread (Mandelstam and Tamm), so the number of distinguishable changes of state by time t satisfies

$$N(t) \leq c E t, \quad c = \frac{2}{\pi \hbar}.$$

In the kernel c is any natural-number upper bound for $2/(\pi \hbar)$ in the chosen units, and the deduction does not depend on its value. The energy E of the speed limit, the mean energy above the ground state, is the field energy of Physical; the finite energy of Section 2, the L^2 class of the datum, is the field energy of Priced; the two are named apart, and the paper claims no map between them. Under the cascade, a hypothesis carried as a field, a singularity by time T requires, before T , more registrations than any given number; its physical reading, that a singularity excites arbitrarily fine scales, is the reader's identification. The two together contradict each other at $N = c E T$. The frame of matter carries its own count, the count of Section 2: blowup by T is a count unbounded by T , and the cascade, a hypothesis carried as a field, the limit-form link between the count and the stages, makes such a count a run of registrations before T above every bound. The speed limit bounds every one of them by $c E T$, so no datum blows up (no_actual_blowup); every count of the frame of matter is then finite at every time, existence acts finitely there, and the frame is regular in the sense of Section 2 (matter_acts_finitely, matter_is_regular). The speed limit carries the weight, and the cascade carries it to the frame's count. The speed limit alone excludes blowup in its limit form: on a world carrying the speed limit and nothing else, no cascade field, the stage counts before a horizon never pass every bound (no_limit_blowup_from_speed_only), and on every physical frame likewise, the cascade entering that deduction nowhere (no_limit_blowup_from_speed); the cascade carries that exclusion to the frame's own count, so that no physical flow blows up and every count is finite at every time (no_physical_blowup, matter_acts_finitely). Both fields are load-bearing. Without the speed limit, blowup in the sense of Section 2 is satisfiable under the cascade, by a world whose count by time one is unbounded, whose frame is not regular, and whose registrations exceed every bound (speed_is_load_bearing); without the cascade the speed limit holds and still a world's count by time one is unbounded and its frame is not regular (cascade_is_load_bearing), that world, speedWorld, its count unbounded at every time and every stage zero by its definition, satisfying Section 2's definition of blowup, so the cascade is a hypothesis a frame can fail and not that definition. The speed limit and the cascade enter as fields of the structure Physical: the speed limit is cited at its grade, and the cascade is a hypothesis carried as a field at its own grade, the limit-form link between the count and the stages, not a cited physical re-

sult and not Section 2's definition, its physical reading part of the reader's identification; a physical frame exists (physical_inhabited), and the deduction from the two fields is on no axiom.

That is the kinetic closure on matter, from the speed limit in two senses: in the limit sense, every stage count before every horizon bounded and so never passing every bound, from the speed limit alone (no_limit_blowup_from_speed_only; on the physical frame, no_limit_blowup_from_speed); in the count sense, no datum blows up and the frame of matter is regular, from the speed limit with the cascade (no_actual_blowup, matter_regular_from_speed). Its scope is matter, a quantum system with a molecular cutoff. The continuum equations idealize matter, and the step from the one to the other is the identification the reader makes in Definition 9.1. It is the step the act supplies.

12. THE RECORD AND THE SEED

No finite record forces the value: for every n the staged world is regular on its first n data and the value fails (finite_record_never_forces). No symmetry lifts the small-data region: the symmetries the text checks, space scaling and viscosity scaling with rotations and translations, keep the dimensionless size $\|u_0\|_{\dot{H}^{1/2}}/\nu$, since space scaling $u_\lambda(x, t) = \lambda u(\lambda x, \lambda^2 t)$ fixes $\dot{H}^{1/2}$ and viscosity scaling multiplies norm and ν alike, so no small datum is carried onto a large one (no_symmetry_lifts). A step uniform in the size reaches every size (uniform_step_forces_all). The seed is evidence for the fibre of Section 6, and for neither of its points.

13. THE FORCED CLASS

The forced alternatives are their own class. The author's transport theorem proves that a forced breakdown transports only to a system that is not closed (Islam 2026d), and his filter paper proves the forced alternative a regularity filter on a prescribed field (Islam 2026c). The unforced closure stands whatever the audit of the September 2026 forced construction finds, and this paper asserts nothing about its correctness.

14. THE GRADE OF THE CLOSURE, STATED WHOLE

The root, universal and undeniable in deed: every denial of it re-enacts it, no outside proof adds to it (denial_reenacts_root, external_proof_adds_nothing), and it holds on every frame once any deed occurs, in the formal sense that a self-grounding root indexed by frame holds on every frame and forces the value on no frame uniformly: no derivation of the value on every frame from the family exists (grounded_on_every_frame_forces_no_value); for that universality it decides no value by itself, neither forcing nor refuting it, both conjuncts of one theorem (undeniable_root_forces_no_value); its undeniable-

ity stays with the root, the reading on the row not being such a family and grounding itself exactly where the value holds (*undeniability_stays_with_the_root*, *row_root_grounds_iff_value*); and read on the row it is the act, the root's own formula on the row (*root_on_row_is_the_act*), keyed, at premise grade (*root_read_on_row_is_keyed*).

Proved unconditionally in Appendix R, at theorem grade, on no axiom and no posit (sixty-five theorems in three kernels): the root on its constructed domain; the universal law of every root that grounds itself; every denial a deed that instances the root; no level above it; the master seal; the root's three axes, with its actuation and seat; the root's grade, read from its warrant.

Proved in core Lean 4, no library, no sorry, no axiom declared, every theorem on no axiom at all (65 theorems): the frame and its two worlds; existence given, its conservativity, its holding in both worlds, its forcing no value; the act, its equality with the value, its key, its exact supply; the closure from the act; the exclusive gates and the universal closure; the refutation form; the energy theorem and the energy record that decides nothing (*energy_record_decides_nothing*); the kinetic theorem, no physical blowup from the speed limit with the cascade and none in the limit form from the speed limit alone (*no_physical_blowup*, *no_limit_blowup_from_speed_only*, and on the physical frame *no_limit_blowup_from_speed*), the load-bearing role of the speed limit and of the cascade (*speed_is_load_bearing*, *cascade_is_load_bearing*), and regularity on matter from the speed limit with the cascade (*matter_regular_from_speed*); the freedom wall in the Bool model and on frames (*record_wall*, *record_wall_on_frames*), the fibre, one free orbit, the prime's shape and its one swap orbit; the two lock counts; the finite record, the symmetry theorem and the uniform step; and the closure whole; the root satisfiable on every background and on its constructed domain (*root_satisfiable*, *constructed_root_holds*); the root read on the row, equal to the act and to the value, and keyed (*root_on_row_is_the_act*, *root_on_row_is_the_value*, *root_on_row_is_keyed*); the act the weakest premise that forces the value (*act_is_the_weakest_premise*); nothing given forces the value or refutes the act on every frame (*nothing_given_forces*, *nothing_given_refutes*); the root grounded on every frame forcing no value, and the row's reading grounding itself exactly where the value holds (*grounded_on_every_frame_forces_no_value*, *undeniability_stays_with_the_root*, *row_root_grounds_iff_value*); the lock, binding the constructed root, the root by its deed, the root read on the row, the keyed act, the closure, the record wall and the finite record in one theorem (*the_lock*), and closed at Appendix R's root with no hypothesis (*the_lock_on_the_root*, *RA₀_is_the_constructed_root*,

regularity_from_root_on_row); the sense of existence alone (*from_existence_alone*).

Carried as fields: the quantum speed limit, cited; and the cascade, a hypothesis carried as a field, the limit-form link between the count and the stages, an unbounded count by a horizon having stage counts before it above every bound, proved load-bearing (*cascade_is_load_bearing*); it is not a cited physical result and not Section 2's definition. The kinetic theorem reads the two in two senses: no blowup in the limit sense, stage counts before a horizon passing every bound, from the speed limit alone, on a world carrying the speed limit and nothing else (*no_limit_blowup_from_speed_only*) and on the physical frame (*no_limit_blowup_from_speed*); no blowup in the count sense, the frame's count by a horizon unbounded, from the speed limit with the cascade, which ties the count to the stages (*no_actual_blowup*); both fields load-bearing for the count sense (*speed_is_load_bearing*, *cascade_is_load_bearing*).

Supplied by the act, named and visible in one input type: existence acting finitely at the continuum, supply, which is the value.

The reader's identification: the frame with the strong solutions of the equations, the analytic blowup of a solution with an unbounded count, and matter with the continuum.

The grade of the closure is the grade of its act, and the act, read on every frame, is the value, which the kernel proves no statement given uniformly on every frame can be. That is why the act is named rather than derived: a derivation of it on every frame from what is given uniformly would contradict *given_is_not_the_value*, and the kernel would not compile.

15. OBJECTIONS, ANSWERED

The act is only a premise. It is the root's own formula read on the row, with the row's existents and the row's actuation (*root_on_row_is_the_act*). The root is universal and undeniable in deed: every denial re-enacts it and no outside proof adds to it (*denial_reenacts_root*, *external_proof_adds_nothing*); it holds in both worlds and so decides no value, neither forcing nor refuting it, both conjuncts of one theorem (*undeniable_root_forces_no_value*). That undeniability stays with the root and does not pass to the reading on the row, which grounds itself exactly where the value holds (*undeniability_stays_with_the_root*, *row_root_grounds_iff_value*); that is why the act is keyed and decides what the root alone cannot (*root_read_on_row_is_keyed*). Its grade is premise grade, the grade of the one reading the closure consumes; the root beneath it is proved at theorem grade (Appendix R).

The act is the conclusion. It is, by *act_is_the_value*, and the paper proves it must be: every premise that

forces the value on every frame implies the act (`act_is_the_weakest_premise`). A closure from a weaker premise is impossible, no statement given on every frame is the value (`given_is_not_the_value`), and a closure from a stronger premise assumes more than the question asks. This is the shape of every closure of a value that differs between coherent worlds, and the paper states it as a theorem rather than discovering it as a defect.

Existence is universally given; why is it not enough? Because what is universally given holds in the blowup world too (`existence_as_given_in_both_worlds`). The universality of existence is exactly what keeps it from choosing between worlds. Existence chooses when it is read on the row, and that reading is the act.

The kinetic theorem is physics. It is a theorem from one cited physical field, the speed limit, which alone excludes blowup in the limit sense, on a world carrying it and nothing else (`no_limit_blowup_from_speed_only`), and one hypothesis carried as a field, the cascade, the limit-form link between the count and the stages, not a cited physical result and not Section 2's definition, with which it excludes blowup in the count sense (`no_actual_blowup`), with a deduction on no axiom. Its conclusion is about matter. The paper does not transport it to the continuum by theorem; it names the transport as the act.

The frame is a skeleton, not the equations. Correct, and stated in Definition 9.1. Every theorem that ranges over frames holds on every frame, so it holds on whichever frame the reader identifies with the equations.

16. FALSIFIERS

F-Blowup. A smooth unforced datum of finite energy whose solution is proved to become singular in finite time. It refutes the act by `blowup_refutes`, and it is the one channel the closure leaves open.

F-Speed. A physical flow whose number of distinguishable changes of state by time t exceeds the speed-limit bound for its energy. It refutes the field speed on that flow, and with it the application of the kinetic theorem to that flow; the theorem, a deduction on no axiom, stands.

F-Cone. The kernel, compiled on a stock toolchain, prints any axiom in any cone, or fails to compile. It refutes the claim that every theorem is on no axiom. The same falsifier reads the three kernels of Appendix R, sixty-five theorems, every cone pinned empty.

17. POSITIONING

The relation words are used as defined: *extends* where a prior result is carried and a theorem is added on it; *bounds* where its reach is stated exactly; *adjacent* where no formal engagement is claimed; *retires* (*premise of*) where the prior results stand and the premise they named is replaced by the act of this paper.

18. CONCLUSION

Existence was given before the question was asked, and the paper proves exactly what that givenness carries: under the root, the form of the closure on every frame (`given_carries_the_form`); nothing beyond what holds without the root (`root_conservative`); and the root, the arrow and the bit of freedom holding in both worlds (`existence_as_given_in_both_worlds`). It proves that existence read on the row is the value; that the value follows from that reading by one act; that nothing escapes the act; that one blowup is the only refuter; and that on matter the speed limit makes the absence of blowup a theorem, alone in the limit sense (`no_limit_blowup_from_speed_only`) and with the cascade, a hypothesis carried as a field, in the count sense (`no_actual_blowup`). The freedom of the row is one bit, walled; in the Bool model it is one free orbit with the shape of a prime's fibre; and three axes lock it. Every theorem stands on no axiom at all.

The verdict, in the words of Section 1, unchanged: the Navier–Stokes regularity problem is closed to one act of existence. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the regular and the blowup world alike. Existence read on the row, that whatever exists acts finitely at every time, is the value, exactly. It is supplied by one act, at premise grade, as one field of one type, and the compiler prints global regularity from it on no axiom. Nothing escapes the act; one blowup is the only refuter. On matter, that no flow blows up is a theorem of the quantum speed limit in two senses: in the limit sense of blowup, stage counts before a horizon passing every bound, from the speed limit alone, on a world carrying the speed limit and nothing else (`no_limit_blowup_from_speed_only`) and on the physical frame (`no_limit_blowup_from_speed`); and in the count sense, the frame's count by a horizon unbounded, from the speed limit with the cascade, a hypothesis carried as a field, the limit-form link between the count and the stages, an unbounded count by a horizon having stage counts before it above every bound, not a cited physical result and not Section 2's definition (`no_actual_blowup`), both fields load-bearing for the count sense (`speed_is_load_bearing`, `cascade_is_load_bearing`). Every theorem of the kernel depends on no axiom at all.

APPENDIX A · RECEIPTS

The kernel, `NS_Existence_Closure.lean`, compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0 (de Moura and Ullrich 2021). It carries sixty-five theorems, and every one prints *does not depend on any axioms*, pinned by `#guard_msgs` so that a drifted cone

Position	What it holds	What this paper does with it	Relation
Leray 1934	weak solutions; the energy inequality	the inequality proved not to decide	bounds
Fujita and Kato 1964	small critical data are global	the small region shown not to lift by symmetry	bounds
Mandelstam and Tamm 1945; Margolus and Levitin 1998	the quantum speed limit	carried as a field; regularity on matter deduced (matter_regular_from_speed)	extends
Tao 2016	an averaged equation blows up	the energy theorem exhibits on the frame what that construction shows analytically	adjacent
Islam 2026g, the Riemann closure	one bit, closed by one act	the template carried to this row	extends
Islam 2026a–f, Navier–Stokes papers	criteria, a premise, closed routes, a witness	the premise retired; the row reduced to existence	retires (premise of)

Table 3 | Positioning. No prior position is contradicted.

fails the compile. Its SHA-256 is

959452e82bd6e3c632ff6d20676568dc
6102d5572bf67e53b531850e3b3ee7c2

The root kernel of Appendix R, T0E_Zero.lean, compiles with exit 0 and no message on both toolchains; its fifteen theorems print *does not depend on any axioms*, pinned by #guard_msgs. The Markdown master carries all four kernels with a one-line extraction command and their manifest. The two further kernels of Appendix R, Triaxial_Actuation.lean and Root_Grade_Ledger.lean, compile with exit 0 and no message on both toolchains; their thirty-two and eighteen theorems print *does not depend on any axioms*, pinned by #guard_msgs; their SHA-256 digests, 566615a6b9610fc9... and c18b68b29d2b6be5..., stand in full in the manifest beside the two others and in Appendix R, and the master carries all four kernels.

APPENDIX B • THE KERNEL

```
/-
NS_Existence_Closure.lean · the kernel of A Formal Completed Proof
of the Navier–Stokes Closure
from Existence Alone

The unforced Navier–Stokes row closed on existence alone, the
freedom arrow beside it. Every theorem
of this file is on no axiom at all: no propext, no Quot.sound, no
Classical.choice.
I The frame, the value, the two coherent worlds.
II Existence as given: the root, the arrow, the bare freedom
bit; what they carry on every
frame, and that they force no value.
III Existence read on the row: the act, equal to the value; the
root read on the row is the act and
the value; nothing given forces or refutes the act; the
closure by one act; nothing escapes;
every instant lands; one blowup refutes; existence as given
carries this form on every frame.
IV The price: energy does not decide, and a regular priced
frame with the blowup frame's energy record
shows the record decides nothing; on matter the speed limit
and the cascade give the frame's own
count finite at every time, regular in the sense of I, with
no physical blowup and no limit blowup
from the speed limit, the limit form holding on a world
carrying the speed limit alone; the speed
limit is load-bearing, and so is the cascade.
V Freedom: one free orbit, walled, in the Bool model and on
frames; the prime's shape, its fibre one
swap orbit.
VI The triaxial lock.
VII The record and the seed: no finite record forces the value
and no symmetry lifts a decided
datum; a uniform step decides every rung of a ladder that
carries it.
VIII The closure, whole.
Core Lean 4, no import, no axiom declared, no sorry. Cones pinned
at the foot.
```

```
ROOT The root, undeniable in deed, and the lock: the root holds
by its deed in both worlds and forces
the value neither way; a root grounded on every frame forces
no value; the undeniability stays with
the root, the root read on the row grounding exactly where
the value holds; Appendix R's constructed
root, repeated verbatim, is the constructed root, and its
lock on the row (the_lock_on_the_root);
read on the row it is the act; one theorem on no axiom binds
the root, its readings, the act, the
value, their keying, the closing theorem, the record wall
and the finite record (the_lock); and one
fixes the sense of 'from existence alone'
(from_existence_alone).
-/

namespace NSClose

/-- ## I · the frame -/

/-- A flow frame: data, and the registrations each datum's flow has
made by time t,
'none' when that count is unbounded by then, which is blowup by that
time. -/
structure Frame where
D : Type
count : D → Nat → Option Nat

/-- A datum is regular when its flow has acted finitely at every
time. -/
def Regular (F : Frame) (d : F.D) : Prop := ∀ t, ∃ n, F.count d t =
some n

/-- The value of the row on a frame: every datum regular. -/
def Value (F : Frame) : Prop := ∀ d, Regular F d

/-- The regular world. -/
def calm : Frame := (Unit, fun _ => some 0)
/-- The blowup world. -/
def burst : Frame := (Unit, fun _ t => match t with | 0 => some 0 |
_ + 1 => none)

theorem calm_value : Value calm := fun _ => {0, rfl}

theorem burst_fails : ¬ Value burst := fun h => match h () 1 with
| (_, hn) => Option.noConfusion hn

/-- ## II · existence as given -/

/-- The root, in its formal reading: to exist is to actuate. -/
def Root (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

/-- The root holds on the empty background, vacuously. -/
theorem root_given : Root Empty (fun e => nomatch e) := fun e =>
nomatch e

/-- What follows from the root uniformly in its symbols holds
without it. -/
theorem root_conservative (S : Prop) (h : ∀ (U : Type) (ΔE : U →
Int), Root U ΔE → S) : S :=
h Empty (fun e => nomatch e) root_given

/-- The arrow is given on every type. -/
def Arrow : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a

theorem arrow_given : Arrow := fun _ => {id, fun _ => rfl}

/-- The freedom bit is given: one orbit of two, off the seat. -/
```

```

theorem freedom_given :  $\forall w : \text{Bool}, (!w) \neq w$ 
| true => Bool.noConfusion
| false => Bool.noConfusion

/-- What is given on every frame forces no value: no statement
reading the same on every frame
is the value. -/
theorem given_is_not_the_value (P : Prop) :  $\neg \forall F : \text{Frame}, (P \leftrightarrow \text{Value } F) :=$ 
  fun h => burst_fails ((h burst).mp ((h calm).mpr calm_value))

theorem arrow_forces_nothing :  $\neg \forall F : \text{Frame}, \text{Arrow} \rightarrow \text{Value } F :=$ 
  fun h => burst_fails (h burst arrow_given)

/-- The given holds in both worlds. -/
theorem given_in_both_worlds : (Arrow  $\wedge$  Value calm)  $\wedge$  (Arrow  $\wedge$   $\neg$ 
  Value burst) :=
  (arrow_given, calm_value), (arrow_given, burst_fails))

/-! ## III · existence read on the row: the act -/

/-- Existence read on the row: whatever exists acts finitely at
every time. -/
def ActsFinitely (F : Frame) : Prop :=  $\forall d \ t, \exists n, F.\text{count } d \ t =$ 
  some n

/-- The act is the value, exactly. -/
theorem act_is_the_value (F : Frame) : ActsFinitely F  $\leftrightarrow$  Value F :=
  (fun h d t => h d t, fun h d t => h d t)

/-- The act is keyed: it holds in one world and fails in the other.
-/
theorem act_is_keyed : ActsFinitely calm  $\wedge$   $\neg$  ActsFinitely burst :=
  (calm_value, burst_fails)

/-! ## The root read on the row -/

/-- The indicator of a decided proposition: one where it holds, zero
where it fails. -/
def indicator {p : Prop} : Decidable p  $\rightarrow$  Int
| isTrue _ => 1
| isFalse _ => 0

/-- The indicator is positive exactly where its proposition holds.
-/
theorem indicator_pos {p : Prop} (h : Decidable p) :  $0 < \text{indicator } h$ 
 $\leftrightarrow$  p :=
  match h with
| isTrue hp => (fun _ => hp, fun _ => show (0 : Int) < 1 by decide)
| isFalse hn => (fun h0 => absurd h0 (show  $\neg$  (0 : Int) < 0 by
  decide), fun hp => absurd hp hn)

/-- Whether an option carries a value, decided. -/
def suppliedDec (o : Option Nat) : Decidable ( $\exists n, o = \text{some } n$ ) :=
  match o with
| some n => isTrue (n, rfl)
| none => isFalse (fun (_, h) => Option.noConfusion h)

/-- THE ROOT READ ON THE ROW: to exist is to actuate, `Root` itself,
with the row's existents, each datum at
each time, and the row's actuation, one where the datum's count is
finite and zero where it is not. -/
def RootOnRow (F : Frame) : Prop :=
  Root (F.D  $\times$  Nat) (fun x => indicator (suppliedDec (F.count x.1
  x.2)))

/-- THE ROOT IS SATISFIABLE ON EVERY BACKGROUND: on every type, with
actuation one at every existent. -/
theorem root_satisfiable (U : Type) : Root U (fun _ => 1) := fun _
=> show (0 : Int) < 1 by decide

/-- THE CONSTRUCTED ROOT, in the words of Appendix R: one existent,
whose actuation is one. A theorem
with no hypothesis and no axiom. -/
theorem constructed_root_holds : Root Unit (fun _ => 1) :=
  root_satisfiable Unit

/-- THE ROOT READ ON THE ROW IS THE ACT. -/
theorem root_on_row_is_the_act (F : Frame) : RootOnRow F  $\leftrightarrow$ 
  ActsFinitely F :=
  (fun h d t => (indicator_pos _).mp (h (d, t)), fun h x =>
    (indicator_pos _).mpr (h x.1 x.2))

/-! THE ROOT READ ON THE ROW IS THE VALUE. -/

```

```

theorem root_on_row_is_the_value (F : Frame) : RootOnRow F  $\leftrightarrow$  Value F
:=
  (fun h => (act_is_the_value F).mp ((root_on_row_is_the_act F).mp
  h),
  fun h => (root_on_row_is_the_act F).mpr ((act_is_the_value F).mpr
  h))

/-- THE ROOT READ ON THE ROW IS KEYED: it holds where the value
holds and fails where the value fails. -/
theorem root_on_row_is_keyed : RootOnRow calm  $\wedge$   $\neg$  RootOnRow burst :=
  ((root_on_row_is_the_value calm).mpr calm_value, fun h =>
  burst_fails ((root_on_row_is_the_value burst).mp h))

/-- THE ACT IS THE WEAKEST PREMISE: the act forces the value on
every frame, and every premise that forces
the value on every frame implies the act. -/
theorem act_is_the_weakest_premise :
  ( $\forall F : \text{Frame}, \text{ActsFinitely } F \rightarrow \text{Value } F$ )  $\wedge$ 
   $\forall Q : \text{Frame} \rightarrow \text{Prop}, (\forall F : \text{Frame}, Q F \rightarrow \text{Value } F) \rightarrow \forall F : \text{Frame}, Q$ 
  F  $\rightarrow$  ActsFinitely F :=
  (fun F h => (act_is_the_value F).mp h, fun _ hQ F hq =>
  (act_is_the_value F).mpr (hQ F hq))

/-- EXISTENCE AS GIVEN HOLDS IN BOTH WORLDS: the root satisfiable on
every background, the arrow and the
freedom bit hold in the world where the value holds and in the world
where it fails. -/
theorem existence_as_given_in_both_worlds :
  (( $\forall U : \text{Type}, \text{Root } U$  (fun _ => 1))  $\wedge$  Arrow  $\wedge$  ( $\forall w : \text{Bool}, (!w) \neq$ 
  w))  $\wedge$  Value calm  $\wedge$   $\neg$  Value burst :=
  ((root_satisfiable, arrow_given, freedom_given), calm_value,
  burst_fails)

/-- NOTHING GIVEN REFUTES THE ACT: no statement that holds refutes
the act on every frame. -/
theorem nothing_given_refutes :  $\forall P : \text{Prop}, P \rightarrow \neg \forall F : \text{Frame}, P \rightarrow$ 
  ActsFinitely F :=
  fun _ hp h => h calm hp act_is_keyed.1

/-- NOTHING GIVEN FORCES THE VALUE: no statement that holds forces
the value on every frame. -/
theorem nothing_given_forces :  $\forall P : \text{Prop}, P \rightarrow \neg \forall F : \text{Frame}, P \rightarrow$ 
  Value F :=
  fun _ hp h => burst_fails (h burst hp)

/-- THE ONLY REFUTER: wherever the act fails, it cannot be that no
instance fails, a datum blowing up. -/
theorem only_refuter (F : Frame) :  $\neg$  ActsFinitely F  $\rightarrow \neg \exists d \ t,$ 
  F.count d t = none :=
  fun hn hne => hn (fun d t => match hc : F.count d t with
  | some n => (n, rfl)
  | none => absurd (d, t, hc) hne)

/-- The actual flows, with the one act of the paper. -/
structure ActualFlows where
  F : Frame
  supply : ActsFinitely F

/-- GLOBAL REGULARITY FROM EXISTENCE, BY ONE ACT. -/
theorem regularity_from_existence (A : ActualFlows) : Value A.F :=
  (act_is_the_value A.F).mp A.supply

/-- THE SUPPLY IS EXACT: an act exists on a frame exactly when the
value holds there. -/
theorem supply_iff (F : Frame) : Nonempty { A : ActualFlows // A.F =
  F }  $\leftrightarrow$  Value F :=
  (fun (A, hA) => hA  $\triangleright$  regularity_from_existence A,
  fun h => (((F, (act_is_the_value F).mpr h), rfl)))

/-- Every instant lands on exactly one gate: a finite count or an
unbounded one. -/
theorem every_instant_lands (F : Frame) (d : F.D) (t : Nat) :
  ( $\exists n, F.\text{count } d \ t = \text{some } n$ )  $\vee$  F.count d t = none :=
  match F.count d t with
| some n => 0r.inl (n, rfl)
| none => 0r.inr rfl

theorem gates_exclusive (F : Frame) (d : F.D) (t : Nat) :
   $\neg$  (( $\exists n, F.\text{count } d \ t = \text{some } n$ )  $\wedge$  F.count d t = none) :=
  fun (⟦_, h⟧, h') => Option.noConfusion (h'.symm.trans h)

/-! NOTHING ESCAPES the act: every datum, at every time, acts
finitely. -/

```

```

theorem nothing_escapes (A : ActualFlows) :  $\forall d t, \exists n, A.F.count d t = some n := A.supply$ 

/-- One blowup refutes the act. -/
theorem blowup_refutes (F : Frame) (d : F.D) (t : Nat) (h : F.count d t = none) :
   $\neg ActsFinitely F :=$ 
  fun hA => match hA d t with
  | (_, hn) => Option.noConfusion (h.symm.trans hn)

/-- EXISTENCE AS GIVEN CARRIES THE FORM: under any root, on every
frame, the act is the value, every instant
lands on a gate, and one blowup refutes the act. -/
theorem given_carries_the_form :  $\forall (U : Type) (\Delta E : U \rightarrow Int), Root U \Delta E \rightarrow$ 
  ( $\forall F : Frame, ActsFinitely F \leftrightarrow Value F$ )  $\wedge$ 
  ( $\forall (F : Frame) (d : F.D) (t : Nat), (\exists n, F.count d t = some n) \vee F.count d t = none$ )  $\wedge$ 
  ( $\forall (F : Frame) (d : F.D) (t : Nat), F.count d t = none \rightarrow \neg ActsFinitely F$ ) :=
  fun _ _ => (act_is_the_value, every_instant_lands,
  blowup_refutes)

/-! ## IV · the price -/

/-- A priced frame with the energy inequality: dissipation paid
never exceeds the energy. -/
structure Priced where
  F      : Frame
  energy  : F.D  $\rightarrow$  Nat
  paid    : F.D  $\rightarrow$  Nat  $\rightarrow$  Nat
  inequality :  $\forall d t, paid d t \leq energy d$ 

def leray : Priced := (burst, fun _ => 1, fun _ _ => 1, fun _ _ =>
  Nat.le_refl 1)

/-- Energy does not decide: the energy inequality holds and a datum
blows up. -/
theorem energy_does_not_decide : ( $\forall d t, leray.paid d t \leq$ 
  leray.energy d)  $\wedge \neg Value leray.F :=$ 
  (leray.inequality, burst_fails)

/-- The regular world, priced: the same energy and the same
dissipation as the blowup frame. -/
def calmPriced : Priced := (calm, fun _ => 1, fun _ _ => 1, fun _ _
=> Nat.le_refl 1)

/-- THE ENERGY RECORD DECIDES NOTHING: a regular priced frame and
the blowup priced frame carry the same energy
and the same dissipation, so no reading of that record is the value
on both. -/
theorem energy_record_decides_nothing :
  ( $\forall d : Unit, calmPriced.energy d = leray.energy d$ )  $\wedge$ 
  ( $\forall (d : Unit) (t : Nat), calmPriced.paid d t = leray.paid d t$ )  $\wedge$ 
  Value calmPriced.F  $\wedge \neg Value leray.F \wedge$ 
   $\neg \exists g : (Unit \rightarrow Nat) \rightarrow (Unit \rightarrow Nat \rightarrow Nat) \rightarrow Prop,$ 
  (g calmPriced.energy calmPriced.paid  $\leftrightarrow$  Value calmPriced.F)  $\wedge$ 
  (g leray.energy leray.paid  $\leftrightarrow$  Value leray.F) :=
  (fun _ => rfl, fun _ _ => rfl, calm_value, burst_fails,
  fun (_, h1, h2) => burst_fails (h2.mp (h1.mpr calm_value)))

/-- A physical frame. Time before a horizon T is resolved by
instants indexed by k, each before T.
`stage d T k` counts the registrations of datum d by the k-th
instant before T, and `count d T` is the frame's
count of datum d by T in the sense of Section I: `none` when it is
unbounded, which is blowup by T. The quantum
speed limit is a cited field: no stage count before T exceeds c·E·T.
The cascade is a hypothesis carried as a
field, the limit-form link between the count and the stages: an
unbounded count by T has a stage count before T
above every bound. -/
structure Physical where
  D      : Type
  energy  : D  $\rightarrow$  Nat
  stage   : D  $\rightarrow$  Nat  $\rightarrow$  Nat  $\rightarrow$  Nat
  c       : Nat
  speed   :  $\forall d T k, stage d T k \leq c * energy d * T$ 
  count   : D  $\rightarrow$  Nat  $\rightarrow$  Option Nat
  cascade :  $\forall d T, count d T = none \rightarrow \forall N, \exists k, N < stage d T k$ 

/-- The frame of matter: the physical frame's own count. -/
def Physical.frame (P : Physical) : Frame := (P.D, P.count)

```

```

/-- Blowup by T on matter, in the sense of Section I: the count by T
is unbounded. -/
def Physical.Blowup (P : Physical) (d : P.D) (T : Nat) : Prop :=
  P.count d T = none

/-- A physical frame exists: one datum of energy one, no
registration, count zero. -/
def matterWorld : Physical :=
  (Unit, fun _ => 1, fun _ _ => 0, 1, fun _ _ => Nat.zero_le _,
  fun _ _ => some 0,
  fun _ _ h => Option.noConfusion h)

theorem physical_inhabited : Nonempty Physical := (matterWorld)

/-- NO ACTUAL FLOW BLOWS UP: finite energy bounds every stage count
before T by c·E·T, and an unbounded count by
T needs stage counts before T above every bound. -/
theorem no_actual_blowup (P : Physical) (d : P.D) (T : Nat) :  $\neg$ 
  P.Blowup d T :=
  fun hb => match P.cascade d T hb (P.c * P.energy d * T) with
  | (k, hlt) => Nat.lt_irrefl _ (Nat.lt_of_lt_of_le hlt (P.speed d
  T k))

/-- NO PHYSICAL BLOWUP: on matter, no datum's count by any horizon
is unbounded. -/
theorem no_physical_blowup (P : Physical) (d : P.D) (T : Nat) :  $\neg$ 
  P.Blowup d T :=
  no_actual_blowup P d T

/-- NO LIMIT BLOWUP FROM THE SPEED LIMIT: on matter, the stage
counts before a horizon never pass every bound. -/
theorem no_limit_blowup_from_speed (P : Physical) (d : P.D) (T :
  Nat) :  $\neg \forall N, \exists k, N < P.stage d T k :=$ 
  fun h => match h (P.c * P.energy d * T) with
  | (k, hlt) => Nat.lt_irrefl _ (Nat.lt_of_lt_of_le hlt (P.speed d
  T k))

/-- The cascade without the speed limit: the same count and the same
cascade hypothesis, no bound. -/
structure CascadeOnly where
  D      : Type
  stage   : D  $\rightarrow$  Nat  $\rightarrow$  Nat  $\rightarrow$  Nat
  count   : D  $\rightarrow$  Nat  $\rightarrow$  Option Nat
  cascade :  $\forall d T, count d T = none \rightarrow \forall N, \exists k, N < stage d T k$ 

def cascadeWorld : CascadeOnly :=
  (Unit, fun _ _ k => k, fun _ _ => none, fun _ _ _ N => (N + 1,
  Nat.lt_succ_self N))

/-- The frame of the cascade world: its own count. -/
def cascadeFrame : Frame := (cascadeWorld.D, cascadeWorld.count)

/-- THE SPEED LIMIT IS LOAD-BEARING: without it blowup is
satisfiable under the cascade; the cascade world's count by
time one is unbounded, its frame is not regular, and its stages
exceed every bound the speed limit would impose. -/
theorem speed_is_load_bearing :
  cascadeWorld.count () 1 = none  $\wedge \neg Value cascadeFrame \wedge \forall b :$ 
  Nat,  $\exists k, \neg cascadeWorld.stage () 1 k \leq b :=$ 
  (rfl, fun h => match h () 1 with | (_, hn) => Option.noConfusion
  hn,
  fun b => (b + 1, Nat.not_succ_le_self b))

/-- The speed limit without the cascade: the same energy, stages,
bound and count, no cascade hypothesis. -/
structure SpeedOnly where
  D      : Type
  energy  : D  $\rightarrow$  Nat
  stage   : D  $\rightarrow$  Nat  $\rightarrow$  Nat  $\rightarrow$  Nat
  c       : Nat
  speed   :  $\forall d T k, stage d T k \leq c * energy d * T$ 
  count   : D  $\rightarrow$  Nat  $\rightarrow$  Option Nat

/-- A speed-limited world: one datum of energy one, no registration,
and a count unbounded at every time. -/
def speedWorld : SpeedOnly :=
  (Unit, fun _ => 1, fun _ _ => 0, 1, fun _ _ => Nat.zero_le _,
  fun _ _ => none)

/-- The frame of the speed-limited world: its own count. -/
def speedFrame : Frame := (speedWorld.D, speedWorld.count)

/-- NO LIMIT BLOWUP FROM THE SPEED LIMIT ALONE: on a world carrying
the speed limit and nothing else, no cascade

```

```

field, the stage counts before a horizon never pass every bound. -/
theorem no_limit_blowup_from_speed_only (P : SpeedOnly) (d : P.D) (T
  : Nat) :  $\neg \forall N, \exists k, N < P.stage\ d\ T :=$ 
  fun h => match h (P.c * P.energy d * T) with
  | (k, hlt) => Nat.lt_irrefl _ (Nat.lt_of_lt_of_le hlt (P.speed d
    T k))

/-- THE CASCADE IS LOAD-BEARING: without it the speed limit holds
  and still the count by time one is
  unbounded, and the frame is not regular. -/
theorem cascade_is_load_bearing :
  ( $\forall d\ T\ k, speedWorld.stage\ d\ T\ k \leq speedWorld.c *
    speedWorld.energy\ d * T$ )  $\wedge$ 
  speedWorld.count () 1 = none  $\wedge \neg$  Value speedFrame :=
  (speedWorld.speed, rfl, fun h => match h () 1 with | (_, hn) =>
    Option.noConfusion hn)

/-- On matter the act holds: existence acts finitely, the frame's
  own count finite at every time, from the speed
  limit and the cascade. -/
theorem matter_acts_finitely (P : Physical) : ActsFinitely P.frame
  :=
  fun d t => show  $\exists n, P.count\ d\ t = some\ n$  from
  match hc : P.count d t with
  | some n => (n, rfl)
  | none => absurd hc (no_actual_blowup P d t)

theorem matter_is_regular (P : Physical) : Value P.frame :=
  (act_is_the_value P.frame).mp (matter_acts_finitely P)

/-- REGULARITY OF MATTER FROM THE SPEED LIMIT: on every physical
  frame every stage count before every horizon is
  bounded by c·E·T, the cited field; no datum blows up; and the frame
  of matter is regular in the sense of Section I. -/
theorem matter_regular_from_speed (P : Physical) :
  ( $\forall d\ T, \exists b, \forall k, P.stage\ d\ T\ k \leq b$ )  $\wedge$  ( $\forall d\ T, \neg P.Blowup\ d\ T$ )  $\wedge$ 
  Value P.frame :=
  (fun d T => (P.c * P.energy d * T, P.speed d T), no_actual_blowup
    P, matter_is_regular P)

/-! ## V · freedom -/

/- The kinetic record, the Bool model of the bit;
  'record_wall_on_frames' is the wall on frames. -/
def kineticRecord (_w : Bool) : Nat := 0

/- No reading of the record returns the world, in the Bool model of
  the bit; 'record_wall_on_frames' is the
  wall on frames. -/
theorem record_wall :  $\neg \exists g : Nat \rightarrow Bool, \forall w, g\ (kineticRecord\ w) =$ 
  w :=
  fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (hg true))

/- The record wall on frames: a record of frames that reads the
  regular world and the blowup world alike
  has no reading that is the value on every frame. -/
theorem record_wall_on_frames ( $\alpha$  : Type) (rec : Frame  $\rightarrow \alpha$ ) (h : rec
  calm = rec burst) :
   $\neg \exists g : \alpha \rightarrow Prop, \forall F : Frame, (g\ (rec\ F) \leftrightarrow Value\ F) :=$ 
  fun (_, hg) => burst_fails ((hg burst).mp (h  $\triangleright$  (hg calm).mpr
    calm_value))

/- ONE FREE ORBIT: negation moves every world, and every world is
  the given one or its negation. -/
theorem one_free_orbit : ( $\forall w : Bool, (!w) \neq w$ )  $\wedge \forall w\ v : Bool, v =$ 
  w  $\vee v = !w :=$ 
  (freedom_given, fun w v => match w, v with
  | true, true => Or.inl rfl
  | true, false => Or.inr rfl
  | false, true => Or.inr rfl
  | false, false => Or.inl rfl)

theorem fibre_is_two : ([false, true].filter (fun w => kineticRecord
  w == kineticRecord true)).length = 2 := by
  decide

def mulFibre (n : Nat) : List (Nat  $\times$  Nat) :=
  (List.range (n + 1)).flatMap fun a =>
  (List.range (n + 1)).filterMap fun b => if a * b = n then some
    (a, b) else none

/- The prime's shape: a prime's multiplicative fibre is two points
  off the seat, the count of
  the value fibre. -/

```

```

theorem prime_shape :
  mulFibre 7 = [(1, 7), (7, 1)]  $\wedge$  (mulFibre 7).all (fun p => p.1
    != p.2) = true  $\wedge$ 
  (mulFibre 7).length = ([false, true].filter (fun w =>
    kineticRecord w == kineticRecord true)).length := by
  decide

/-- THE PRIME FIBRE IS ONE SWAP ORBIT: swapping the factors of each
  point reverses the fibre. -/
theorem prime_fibre_one_swap_orbit : (mulFibre 7).map (fun p =>
  (p.2, p.1)) = (mulFibre 7).reverse := by
  decide

/-! ## VI · the triaxial lock -/

def dot2 (r x : Bool  $\times$  Bool  $\times$  Bool) : Bool :=
  xor (r.1 && x.1) (xor (r.2.1 && x.2.1) (r.2.2 && x.2.2))

def cube : List (Bool  $\times$  Bool  $\times$  Bool) :=
  [false, true].flatMap fun a => [false, true].flatMap fun b =>
  [false, true].map fun c => (a, b, c)

def nonzero : List (Bool  $\times$  Bool  $\times$  Bool) := cube.filter (fun r => r
  != (false, false, false))

def solCount (rows : List ((Bool  $\times$  Bool  $\times$  Bool)  $\times$  Bool)) : Nat :=
  (cube.filter (fun x => rows.all (fun rt => dot2 rt.1 x ==
    rt.2))).length

def xor3 (a b : Bool  $\times$  Bool  $\times$  Bool) : Bool  $\times$  Bool  $\times$  Bool :=
  (xor a.1 b.1, xor a.2.1 b.2.1, xor a.2.2 b.2.2)

theorem two_axes_leave_two :
  nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
    [false, true].all (fun t1 => [false, true].all (fun t2 =>
      solCount [(r1, t1), (r2, t2)] == 2)))) = true := by decide

theorem three_axes_lock_one :
  nonzero.all (fun r1 => nonzero.all (fun r2 => nonzero.all (fun
    r3 =>
      r1 == r2 || r3 == r1 || r3 == r2 || r3 == xor3 r1 r2 ||
      [false, true].all (fun t1 => [false, true].all (fun t2 =>
        solCount [(r1, t1), (r2, t2), (r3, t3)] == 1)))))) = true :=
  by decide

/-! ## VII · the record and the seed -/

/- The world regular on its first n data and blowing up after. -/
def staged (n : Nat) : Frame :=
  (Nat, fun d t => if d < n then some 0 else match t with | 0 =>
    some 0 | _ + 1 => none)

/- No finite record forces the value: the first n data are regular
  and the value fails. -/
theorem finite_record_never_forces (n : Nat) :
  ( $\forall d, d < n \rightarrow Regular\ (staged\ n)\ d$ )  $\wedge \neg$  Value (staged n) :=
  (fun d hd t => {0, by
    show (if d < n then some 0 else match t with | 0 => some 0 | _
      + 1 => none) = some 0
    rw [if_pos hd]],
  fun h => match h n 1 with
  | (_, hk) => by
    have e : (staged n).count n 1 = none := by
      show (if n < n then some 0 else (none : Option Nat)) = none
      rw [if_neg (Nat.lt_irrefl n)]
    exact Option.noConfusion (e.symm.trans hk)})

/- A symmetry keeps the size; the small region is a union of
  orbits. -/
structure Symmetric (D : Type) (size : D  $\rightarrow$  Nat) where
  Grp : Type
  act : Grp  $\rightarrow$  D  $\rightarrow$  D
  keep :  $\forall g\ d, size\ (act\ g\ d) = size\ d$ 

theorem no_symmetry_lifts {D : Type} (size : D  $\rightarrow$  Nat) (S : Symmetric
  D size) (r : Nat) (d e : D)
  (hd : size d  $\leq$  r) (he : r < size e) :  $\forall g, S.act\ g\ d \neq e :=$ 
  fun g h =>
  have k : size e = size d := h  $\triangleright$  S.keep g d
  have h2 : size e  $\leq$  r := by rw [k]; exact hd
  Nat.lt_irrefl r (Nat.lt_of_lt_of_le he h2)

/- A ladder in the size, with a uniform step, reaches every size. -/

```

```

structure Ladder where
  Bounded : Nat → Prop
  base    : Bounded 0
  mono    : ∀ r s, Bounded s → r ≤ s → Bounded r
  δ       : Nat → Nat
  step    : ∀ r, Bounded r → Bounded (r + δ r)

theorem uniform_step_forces_all (L : Ladder) (hδ : ∀ r, 1 ≤ L.δ r) :
  ∀ r, L.Bounded r
| 0 => L.base
| r + 1 => L.mono (r + 1) (r + L.δ r) (L.step r
  (uniform_step_forces_all L hδ r))
  (Nat.add_le_add_left (hδ r) r)

/-! ## VIII · the closure, whole -/
theorem the_closure :
  -- existence as given carries the form (given_carries_the_form),
  -- adds nothing (root_conservative),
  -- and forces no value
  (∀ S : Prop, (∀ (U : Type) (ΔE : U → Int), Root U ΔE → S) → S) ∧
  (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) ∧
  ((Arrow ∧ Value calm) ∧ (Arrow ∧ ¬ Value burst)) ∧
  -- existence read on the row is the value, by one act
  (∀ F : Frame, ActsFinitely F ↔ Value F) ∧
  (ActsFinitely calm ∧ ¬ ActsFinitely burst) ∧
  (∀ A : ActualFlows, Value A.F) ∧
  (∀ (A : ActualFlows), ∀ d t, ∃ n, A.F.count d t = some n) ∧
  (∀ (F : Frame) (d : F.D) (t : Nat), (∃ n, F.count d t = some n) →
  F.count d t = none) ∧
  (∀ (F : Frame) (d : F.D) (t : Nat), F.count d t = none →
  ActsFinitely F) ∧
  -- the price
  ((∀ d t, leray.paid d t ≤ leray.energy d) ∧ ¬ Value leray.F) ∧
  (∀ (P : Physical) (d : P.D) (T : Nat), ¬ P.Blowup d T) ∧
  (cascadeWorld.count () 1 = none ∧ ¬ Value cascadeFrame A ∀ b :
  Nat, ∃ k, ¬ cascadeWorld.stage () 1 k ≤ b) ∧
  (∀ P : Physical, Value P.frame) ∧
  -- freedom and the lock
  (¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w) ∧
  (∀ w : Bool, (!w) ≠ w) ∧
  -- the record
  (∀ n, (∀ d, d < n → Regular (staged n) d) ∧ ¬ Value (staged n)) :=
  (root_conservative, given_is_not_the_value, given_in_both_worlds,
  act_is_the_value,
  act_is_keyed, regularity_from_existence, nothing_escapes,
  every_instant_lands, blowup_refutes,
  energy_does_not_decide, no_actual_blowup, speed_is_load_bearing,
  matter_is_regular, record_wall, freedom_given,
  finite_record_never_forces)

/-! ## The root, undeniable in deed -/

/-- A self-grounding root, the structure of Appendix R in the same
words: a type of acts, inhabited, every
one of which instances the root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-- THE ROOT IS UNDENIABLE IN DEED: a denial of a self-grounding
root is a deed, and instances it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R :=
  G.instances denial

/-- No outside proof adds to a self-grounding root: an outside proof
of it is worth exactly the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R) ↔ R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- THE UNDENIABLE ROOT HOLDS IN BOTH WORLDS: it is held by its deed
in a frame where the value holds
and in one where it fails, so it forces the value on no frame
uniformly and refutes it on none uniformly. -/
theorem undeniable_root_forces_no_value {R : Prop} (G :
  SelfGrounding R) :
  R ∧ Value calm ∧ ¬ Value burst ∧ (∀ F : Frame, R → Value F) ∧ ¬
  (∀ F : Frame, R → ¬ Value F) :=
  (G.instances G.anAct, calm_value, burst_fails, fun h =>
  burst_fails (h burst (G.instances G.anAct)),
  fun h => h calm (G.instances G.anAct) calm_value)

/-! A root grounded on every frame forces no value: a family of
roots, each held by its deed, holds on every
frame and does not force the value on every frame. -/
theorem grounded_on_every_frame_forces_no_value (R : Frame → Prop)
  (G : ∀ F : Frame, SelfGrounding (R F)) :
  (∀ F : Frame, R F) ∧ ¬ (∀ F : Frame, R F → Value F) :=
  (fun F => (G F).instances (G F).anAct, fun h => burst_fails (h
  burst ((G burst).instances (G burst).anAct)))

/-- THE CONSTRUCTED ROOT GROUNDS ITSELF: one act, the unit, and
every act instances the constructed root. -/
def constructedGrounding : SelfGrounding (Root Unit (fun _ => 1)) :=
  (Unit, (), fun _ => constructed_root_holds)

/-! THE ROOT READ ON THE ROW GROUNDS ITSELF EXACTLY WHERE THE VALUE
HOLDS: a grounding of the row's reading
exists on a frame exactly when the value holds there. -/
theorem row_root_grounds_iff_value (F : Frame) : Nonempty
  (SelfGrounding (RootOnRow F)) ↔ Value F :=
  (fun (G) => (root_on_row_is_the_value F).mp (G.instances G.anAct),
  fun h => ((Unit, ()), fun _ => (root_on_row_is_the_value F).mpr
  h)))

/-! THE UNDENIABILITY STAYS WITH THE ROOT: the constructed root is
grounded, it does not carry over to the
root read on every frame, and the row's reading is not grounded on
every frame. -/
theorem undeniable_stays_with_the_root :
  Nonempty (SelfGrounding (Root Unit (fun _ => 1))) ∧
  ¬ (∀ F : Frame, Root Unit (fun _ => 1) → RootOnRow F) ∧
  ¬ (∀ F : Frame, Nonempty (SelfGrounding (RootOnRow F))) :=
  ((constructedGrounding), fun h => root_on_row_is_keyed.2 (h burst
  constructed_root_holds),
  fun h => burst_fails ((row_root_grounds_iff_value burst).mp (h
  burst)))

/-! Appendix R's actuation, verbatim: the one existent actuates by
one. -/
def ΔE₀ : Unit → Int := fun _ => 1

/-! Appendix R's constructed root, verbatim: every existent of the
unit background actuates. -/
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-! Appendix R's grounding, verbatim: one act, and every act
instances the constructed root. -/
def ra₀ : SelfGrounding RA₀ := (Unit, ()), fun _ => show (0 : Int)
  < 1 by decide)

/-! APPENDIX R'S ROOT IS THE CONSTRUCTED ROOT: the verbatim root and
the constructed root are one statement. -/
theorem RA₀_is_the_constructed_root : RA₀ ↔ Root Unit (fun _ => 1) :=
  Iff.rfl

/-! REGULARITY FROM THE ROOT ON THE ROW: wherever the root read on
the row holds, the value holds. -/
theorem regularity_from_root_on_row (F : Frame) (h : RootOnRow F) :
  Value F :=
  (root_on_row_is_the_value F).mp h

/-! THE LOCK ON THE ROOT: Appendix R's root holds by its deed,
forces no value on every frame, read on the
row is the value exactly, and the closing theorem gives the value on
every actual frame. -/
theorem the_lock_on_the_root :
  RA₀ ∧ ¬ (∀ F : Frame, RA₀ → Value F) ∧ (∀ F : Frame, RootOnRow F
  ↔ Value F) ∧ (∀ A : ActualFlows, Value A.F) :=
  (ra₀.instances ra₀.anAct, fun h => burst_fails (h burst
  (ra₀.instances ra₀.anAct)), root_on_row_is_the_value,
  regularity_from_existence)

/-! THE ROOT READ ON THE ROW IS KEYED: the root holds, and read
uniformly on the frames it gives no value;
read on the row, with the row's existents and the row's actuation,
it is the value exactly, and keyed. -/
theorem root_read_on_row_is_keyed {R : Prop} (G : SelfGrounding R) :
  R ∧ ¬ (∀ F : Frame, R → Value F) ∧ (∀ F : Frame, RootOnRow F ↔
  Value F) ∧ (RootOnRow calm ∧ ¬ RootOnRow burst) :=
  (G.instances G.anAct, fun h => burst_fails (h burst (G.instances
  G.anAct)), root_on_row_is_the_value,
  root_on_row_is_keyed)

```

```

/-- THE LOCK, one theorem on no axiom: the constructed root holds
with no hypothesis; every self-grounding
root holds by its deed; no given statement is the value on every
frame; the root read uniformly on the frames
gives no value; read on the row the root is the act, the act is the
value exactly, and both are keyed; the
closing theorem gives the value on every actual frame; no reading of
the kinetic record returns the bit; and
no finite record forces the value. -/
theorem the_lock {R : Prop} (G : SelfGrounding R) :
  Root Unit (fun _ => 1) ∧
  R ∧
  (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) ∧
  ¬ (∀ F : Frame, R → Value F) ∧
  (∀ F : Frame, RootOnRow F ↔ ActsFinitely F) ∧
  (∀ F : Frame, ActsFinitely F ↔ Value F) ∧
  (RootOnRow calm ∧ ¬ RootOnRow burst) ∧
  (ActsFinitely calm ∧ ¬ ActsFinitely burst) ∧
  (∀ A : ActualFlows, Value A.F) ∧
  (¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w) ∧
  (∀ n : Nat, (∀ d, d < n → Regular (staged n) d) ∧ ¬ Value
  (staged n)) :=
  (constructed_root_holds, G.instances G.anAct,
  given_is_not_the_value, (root_read_on_row_is_keyed G).2.1,
  root_on_row_is_the_act, act_is_the_value, root_on_row_is_keyed,
  act_is_keyed, regularity_from_existence, record_wall,
  finite_record_never_forces)

/-- FROM EXISTENCE ALONE, in the sense the title carries: the
closing theorem consumes existence read on
the row and nothing else; that reading is available on exactly the
frames where the value holds; and no
statement given without the reading is the value on every frame. One
theorem, on no axiom. -/
theorem from_existence_alone :
  (∀ A : ActualFlows, Value A.F) ∧
  (∀ F : Frame, Nonempty { A : ActualFlows // A.F = F } ↔ Value F) ∧
  (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) :=
  (regularity_from_existence, supply_iff, given_is_not_the_value)

end NSClose

/-! ## Cones, pinned as printed -/
/-- info: 'NSClose.calm_value' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.calm_value
/-- info: 'NSClose.burst_fails' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.burst_fails
/-- info: 'NSClose.root_given' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.root_given
/-- info: 'NSClose.root_conservative' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.root_conservative
/-- info: 'NSClose.arrow_given' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.arrow_given
/-- info: 'NSClose.freedom_given' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.freedom_given
/-- info: 'NSClose.given_is_not_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.given_is_not_the_value
/-- info: 'NSClose.arrow_forces_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.arrow_forces_nothing
/-- info: 'NSClose.given_in_both_worlds' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.given_in_both_worlds
/-- info: 'NSClose.act_is_the_value' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.act_is_the_value
/-- info: 'NSClose.act_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.act_is_keyed
/-- info: 'NSClose.regularity_from_existence' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.regularity_from_existence
/-- info: 'NSClose.supply_iff' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.supply_iff
/-- info: 'NSClose.every_instant_lands' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.every_instant_lands
/-- info: 'NSClose.gates_exclusive' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.gates_exclusive
/-- info: 'NSClose.nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.nothing_escapes
/-- info: 'NSClose.blowup_refutes' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.blowup_refutes

```

```

/-- info: 'NSClose.energy_does_not_decide' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.energy_does_not_decide
/-- info: 'NSClose.no_actual_blowup' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.no_actual_blowup
/-- info: 'NSClose.speed_is_load_bearing' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.speed_is_load_bearing
/-- info: 'NSClose.matter_acts_finitely' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.matter_acts_finitely
/-- info: 'NSClose.matter_is_regular' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.matter_is_regular
/-- info: 'NSClose.record_wall' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.record_wall
/-- info: 'NSClose.fibre_is_two' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.fibre_is_two
/-- info: 'NSClose.prime_shape' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.prime_shape
/-- info: 'NSClose.two_axes_leave_two' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.two_axes_leave_two
/-- info: 'NSClose.three_axes_lock_one' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.three_axes_lock_one
/-- info: 'NSClose.finite_record_never_forces' does not depend on
any axioms -/
#guard_msgs in #print axioms NSClose.finite_record_never_forces
/-- info: 'NSClose.no_symmetry_lifts' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.no_symmetry_lifts
/-- info: 'NSClose.uniform_step_forces_all' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.uniform_step_forces_all
/-- info: 'NSClose.the_closure' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.the_closure
/-- info: 'NSClose.denial_reenacts_root' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.denial_reenacts_root
/-- info: 'NSClose.external_proof_adds_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms NSClose.external_proof_adds_nothing
/-- info: 'NSClose.undeniable_root_forces_no_value' does not depend
on any axioms -/
#guard_msgs in #print axioms NSClose.undeniable_root_forces_no_value
/-- info: 'NSClose.root_read_on_row_is_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.root_read_on_row_is_keyed
/-- info: 'NSClose.the_lock' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.the_lock
/-- info: 'NSClose.from_existence_alone' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.from_existence_alone
/-- info: 'NSClose.indicator_pos' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.indicator_pos
/-- info: 'NSClose.root_satisfiable' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.root_satisfiable
/-- info: 'NSClose.constructed_root_holds' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.constructed_root_holds
/-- info: 'NSClose.root_on_row_is_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.root_on_row_is_the_act
/-- info: 'NSClose.root_on_row_is_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.root_on_row_is_the_value
/-- info: 'NSClose.root_on_row_is_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.root_on_row_is_keyed
/-- info: 'NSClose.act_is_the_weakest_premise' does not depend on
any axioms -/
#guard_msgs in #print axioms NSClose.act_is_the_weakest_premise
/-- info: 'NSClose.existence_as_given_in_both_worlds' does not
depend on any axioms -/
#guard_msgs in #print axioms
  NSClose.existence_as_given_in_both_worlds
/-- info: 'NSClose.nothing_given_refutes' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.nothing_given_refutes
/-- info: 'NSClose.only_refuter' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.only_refuter

```

```

/-- info: 'NSClose.matter_regular_from_speed' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.matter_regular_from_speed
/-- info: 'NSClose.physical_inhabited' does not depend on any axioms
-/
#guard_msgs in #print axioms NSClose.physical_inhabited
/-- info: 'NSClose.nothing_given_forces' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.nothing_given_forces
/-- info: 'NSClose.given_carries_the_form' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.given_carries_the_form
/-- info: 'NSClose.energy_record_decides_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms NSClose.energy_record_decides_nothing
/-- info: 'NSClose.no_physical_blowup' does not depend on any axioms
-/
#guard_msgs in #print axioms NSClose.no_physical_blowup
/-- info: 'NSClose.no_limit_blowup_from_speed' does not depend on
any axioms -/
#guard_msgs in #print axioms NSClose.no_limit_blowup_from_speed
/-- info: 'NSClose.cascade_is_load_bearing' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.cascade_is_load_bearing
/-- info: 'NSClose.no_limit_blowup_from_speed_only' does not depend
on any axioms -/
#guard_msgs in #print axioms NSClose.no_limit_blowup_from_speed_only
/-- info: 'NSClose.record_wall_on_frames' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.record_wall_on_frames
/-- info: 'NSClose.one_free_orbit' does not depend on any axioms -/
#guard_msgs in #print axioms NSClose.one_free_orbit
/-- info: 'NSClose.prime_fibre_one_swap_orbit' does not depend on
any axioms -/
#guard_msgs in #print axioms NSClose.prime_fibre_one_swap_orbit
/-- info: 'NSClose.grounded_on_every_frame_forces_no_value' does not
depend on any axioms -/
#guard_msgs in #print axioms
NSClose.grounded_on_every_frame_forces_no_value
/-- info: 'NSClose.row_root_grounds_iff_value' does not depend on
any axioms -/
#guard_msgs in #print axioms NSClose.row_root_grounds_iff_value
/-- info: 'NSClose.undeniability_stays_with_the_root' does not
depend on any axioms -/
#guard_msgs in #print axioms
NSClose.undeniability_stays_with_the_root
/-- info: 'NSClose.RA0 is the constructed root' does not depend on
any axioms -/
#guard_msgs in #print axioms NSClose.RA0 is the constructed root
/-- info: 'NSClose.regularity_from_root_on_row' does not depend on
any axioms -/
#guard_msgs in #print axioms NSClose.regularity_from_root_on_row
/-- info: 'NSClose.the_lock_on_the_root' does not depend on any
axioms -/
#guard_msgs in #print axioms NSClose.the_lock_on_the_root

```

APPENDIX R • THE ROOT ON NO AXIOM AND NO POSIT

The root on which this paper stands is proved here, in this paper, so that no reader needs another document to check it. The kernel below, R.1, T0E_Zero.lean, is carried verbatim from the root paper (Islam 2026j). Compiled on Lean 4.19.0 and on Lean 4.22.0 it exits 0 with no message: it declares no axiom, imports nothing, and every one of its fifteen theorems prints *does not depend on any axioms*, pinned at the foot of the listing. Five declarations carry its root sections, SelfGrounding, SelfVerifying, ΔE_0 , RA_0 and ra_0 , and all five stand in the listing; the algebra of its master seal rests on the remaining definitions of the listing, the quaternion chart among them. A root grounds itself when deeds occur and every deed instances it (SelfGrounding); the root kernel's own comments call a deed an act. The constructed domain has one exis-

tent, whose energy of actuation is one, so the Root Axiom there reads $\forall x, 0 < \Delta E_0(x)$ (RA_0). On that domain the Root Axiom is a theorem with no axiom and no hypothesis (root_on_the_constructed_domain). For every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds anything to it, one closed theorem on no axiom (the_floor_is_universal). Every denial of the root is a deed that instances it (denial_reenacts_root); the root is held by its deed (seated_undeniable); it is self-verifying (denial_instantiates); and no level stands above it (no_level_above). The master seal binds the root's universal law and the constructed root with the Return, the scalar line as the fixed set of conjugation on the chart, and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4, in one theorem on no axiom (the_master_seal); it states the three counts side by side and no map between them. The body of this paper reads the root on its row; this appendix is the root itself. Its SHA-256 is

```

0cfcd9f6b399ecbaef01e762a4d9f104
90288f1053827b404428062e55076238

```

```

/-
T0E_Zero.lean • Theory of Theories of Everything (T0E of All
T0Es): Existence Proves Existence Only by Motion
The zero-axiom, zero-posit kernel. Core Lean 4: no import, no
axiom declared, no sorry, no native_decide, and
every theorem rests on no axiom at all, not even propositional
extensionality, quotients or choice.
I • The self-grounding root. Acts occur and every act instances
the root. Then every denial is an act and
re-enacts the root, the root is held by the act itself, and no
external proof adds anything to it: the
Omega Boundary, proved for every root that grounds itself,
universally, by quantification.
II • The constructed root. On the constructed domain the Root
Axiom is a theorem.
III • The Return and the line. On the integer quaternions i·j·k =
-1 and each unit squares to -1; conjugation is
an involution and its fixed set on the chart is exactly the
scalar line.
IV • The twelve gates, recovered three ways inside the integers of
the quaternions: the twenty-four Hurwitz units
counted twelve up to sign; the twelve even permutations with
class equation 1, 3, 4, 4; and the norm-two
shell of twenty-four, split twelve and twelve.
V • The regress terminated. Above a self-grounding root no level
grounds it further.
VI • The master seal, every result bound in one theorem.
-/
namespace T0E0

/-- ## I • The self-grounding root: the Omega Boundary -/

/-- A self-grounding root: acts occur, and every act instances the
root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-- A proposition whose denial hands it over. -/
def SelfVerifying (P : Prop) : Prop := ¬P → P

/-- THE OMEGA BOUNDARY: every denial of the root is an act, and
re-enacts it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R := G.instances denial

```

```

/-- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by
the act itself. -/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
  G.instances G.anAct

/-- No external proof adds anything to the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R) ↔ R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- The deed of denying the root hands it over. -/
theorem denial_instantiates {R : Prop} (G : SelfGrounding R) :
  SelfVerifying R := fun _ => G.instances G.anAct

/-- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the
root holds, every act re-enacts it, and no
external proof adds to it. -/
theorem the_floor_is_universal :
  ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q :
  Prop, (Q → R) ↔ R) :=
  fun _ G => (seated_undeniable G, denial_reenacts_root G,
  external_proof_adds_nothing G)

/-! ## II · The constructed root -/

/-- The Root Axiom on the constructed one-point domain: its one
existent actuates, at positive energy. -/
def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed domain grounds its root: its one act is a deed,
and the deed actuates. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int)
  < 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
  ra₀

/-! ## III · The Return and the line -/

structure Q where
  (a b c d : Int)
  deriving DecidableEq

def Q.mul (x y : Q) : Q :=
  (x.a * y.a - x.b * y.b - x.c * y.c - x.d * y.d, x.a * y.b + x.b *
  y.a + x.c * y.d - x.d * y.c,
  x.a * y.c - x.b * y.d + x.c * y.a + x.d * y.b, x.a * y.d + x.b *
  y.c - x.c * y.b + x.d * y.a)

def Q.conj (x : Q) : Q := (x.a, -x.b, -x.c, -x.d)
def qone : Q := (1, 0, 0, 0)
def qneg : Q := (-1, 0, 0, 0)
def qi : Q := (0, 1, 0, 0)
def qj : Q := (0, 0, 1, 0)
def qk : Q := (0, 0, 0, 1)

/-- THE RETURN: i·j·k = -1. -/
theorem the_return : (qi.mul qj).mul qk = qneg := by decide

/-- Each unit squares to -1. -/
theorem units_square_to_minus_one : qi.mul qi = qneg ∧ qj.mul qj =
  qneg ∧ qk.mul qk = qneg := by decide

/-- The chart: every quaternion with coordinates in -2 ... 2. -/
def chart : List Int := [-2, -1, 0, 1, 2]
def quats : List Q :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
    => chart.map fun d => (a, b, c, d)

set_option maxRecDepth 100000 in
/-- Conjugation is an involution on the chart. -/
theorem conj_involution_on_the_chart : quats.all (fun q =>
  q.conj.conj == q) = true := by decide

set_option maxRecDepth 100000 in
/-- THE LINE: conjugation fixes exactly the scalar line on the
chart. -/
theorem the_line_is_the_fixed_set :
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
  q.d == 0)) = true := by decide

/-! ## IV · The twelve gates, recovered three ways -/

```

```

/-- Hurwitz quaternions in doubled coordinates: all even, or all
odd. -/
def hurwitz (a b c d : Int) : Bool :=
  (a % 2 == 0 && b % 2 == 0 && c % 2 == 0 && d % 2 == 0) || (a % 2 !=
  0 && b % 2 != 0 && c % 2 != 0 && d % 2 != 0)
/-- Doubled coordinates -2 ... 2 hold every unit (doubled norm four)
and every element of the norm-two shell (doubled
norm eight), since a doubled coordinate of absolute value three
or more already exceeds both norms. -/
def doubled : List (Int × Int × Int × Int) :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
    => chart.map fun d => (a, b, c, d)

/-- The Hurwitz units: norm one, doubled norm four. -/
def units : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
  4 && hurwitz a b c d

/-- The first nonzero coordinate is positive: one representative of
each pair ±u. -/
def upToSign (p : Int × Int × Int × Int) : Bool :=
  let (a, b, c, d) := p
  if a != 0 then 0 < a else if b != 0 then 0 < b else if c != 0 then
  0 < c else 0 < d

set_option maxRecDepth 100000 in
/-- FIRST WAY: twenty-four Hurwitz units, twelve up to sign. -/
theorem twenty_four_units_twelve_gates : units.length = 24 ∧
  (units.filter upToSign).length = 12 := by decide

/-- The norm-two shell: doubled norm eight. -/
def shell : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
  8 && hurwitz a b c d

set_option maxRecDepth 100000 in
/-- THIRD WAY: the norm-two shell has twenty-four elements, split
twelve and twelve by sign. -/
theorem the_shell_splits_twelve_and_twelve :
  shell.length = 24 ∧ (shell.filter upToSign).length = 12 ∧
  (shell.filter (fun p => !upToSign p)).length = 12 := by
  decide

/-- Permutations of four points, as their lists of images. -/
def perms4 : List (List Nat) :=
  [0, 1, 2, 3].flatMap fun a => [0, 1, 2, 3].flatMap fun b => [0, 1,
  2, 3].flatMap fun c =>
  [0, 1, 2, 3].filterMap fun d => if [a, b, c, d].eraseDups.length
  == 4 then some [a, b, c, d] else none

def inversions (p : List Nat) : Nat :=
  ((List.range 4).flatMap fun i => (List.range 4).map fun j => (i,
  j)).countP
  fun (i, j) => i < j && p.get i > p.get j

/-- The twelve gates as the even permutations. -/
def gates : List (List Nat) := perms4.filter fun p => inversions p %
  2 == 0

def compose (p q : List Nat) : List Nat := q.map fun i => p.get i 0
def inverse (p : List Nat) : List Nat := [0, 1, 2, 3].map fun i =>
  p.idxOf i

def conjClass (g : List Nat) : List (List Nat) := (gates.map fun h
  => compose (compose h g) (inverse h)).eraseDups

def insertSorted (n : Nat) : List Nat → List Nat
  | [] => [n]
  | m :: ms => if n ≤ m then n :: m :: ms else m :: insertSorted n ms

def sortNat (l : List Nat) : List Nat := l.foldr insertSorted []

/-- The class equation from the sorted class size of each element: a
class of size s contributes s elements. -/
def collapse : Nat → List Nat → List Nat
  | 0, _ => []
  | _ + 1, [] => []
  | fuel + 1, s :: rest => s :: collapse fuel (rest.drop (s - 1))

set_option maxRecDepth 100000 in
/-- SECOND WAY: the twelve gates, with class equation 1, 3, 4, 4. -/
theorem the_class_equation :

```

```

    gates.length = 12 ∧ collapse 12 (sortNat (gates.map fun g =>
    (conjClass g).length)) = [1, 3, 4, 4] := by
decide

/-! ## V · The regress terminated -/

/-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a
    further ground of the root is idle; the root needs
    no level above it. -/
theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q :
    Prop, (Q → R) → R := fun _ _ => G.instances G.anAct

/-! ## VI · The master seal -/

/-- THE MASTER SEAL: the floor is universal; the root is a theorem
    on the constructed domain; the Return closes;
    the line is the fixed set; and the twelve gates are recovered
    three ways. -/
theorem the_master_seal :
    (∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q
    : Prop, (Q → R) ↔ R)) ∧
    RA0 ∧ (qi.mul qj).mul qk = qneg ∧
    quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
    q.d == 0)) = true ∧
    (units.filter upToSign).length = 12 ∧ gates.length = 12 ∧
    (shell.filter upToSign).length = 12 ∧
    collapse 12 (sortNat (gates.map fun g => (conjClass g).length))
    = [1, 3, 4, 4] :=
    (the_floor_is_universal, root_on_the_constructed_domain,
    the_return, the_line_is_the_fixed_set,
    twenty_four_units_twelve_gates.2, the_class_equation.1,
    the_shell_splits_twelve_and_twelve.2.1, the_class_equation.2)

end TOE0

/-! ## The zero law, pinned: every cone is empty -/
/-- info: 'TOE0.denial_reenacts_root' does not depend on any axioms
-/
#guard_msgs in #print axioms TOE0.denial_reenacts_root
/-- info: 'TOE0.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.seated_undeniable
/-- info: 'TOE0.external_proof_adds_nothing' does not depend on any
    axioms -/
#guard_msgs in #print axioms TOE0.external_proof_adds_nothing
/-- info: 'TOE0.denial_instantiates' does not depend on any axioms
-/
#guard_msgs in #print axioms TOE0.denial_instantiates
/-- info: 'TOE0.the_floor_is_universal' does not depend on any
    axioms -/
#guard_msgs in #print axioms TOE0.the_floor_is_universal
/-- info: 'TOE0.root_on_the_constructed_domain' does not depend on
    any axioms -/
#guard_msgs in #print axioms TOE0.root_on_the_constructed_domain
/-- info: 'TOE0.the_return' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_return
/-- info: 'TOE0.units_square_to_minus_one' does not depend on any
    axioms -/
#guard_msgs in #print axioms TOE0.units_square_to_minus_one
/-- info: 'TOE0.conj_involutive_on_the_chart' does not depend on any
    axioms -/
#guard_msgs in #print axioms TOE0.conj_involutive_on_the_chart
/-- info: 'TOE0.the_line_is_the_fixed_set' does not depend on any
    axioms -/
#guard_msgs in #print axioms TOE0.the_line_is_the_fixed_set
/-- info: 'TOE0.twenty_four_units_twelve_gates' does not depend on
    any axioms -/
#guard_msgs in #print axioms TOE0.twenty_four_units_twelve_gates
/-- info: 'TOE0.the_shell_splits_twelve_and_twelve' does not depend
    on any axioms -/
#guard_msgs in #print axioms TOE0.the_shell_splits_twelve_and_twelve
/-- info: 'TOE0.the_class_equation' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_class_equation
/-- info: 'TOE0.no_level_above' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.no_level_above
/-- info: 'TOE0.the_master_seal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_master_seal

```

The root's axes are proved in the second kernel, R.2, Triaxial_Actuation.lean (32 theorems, every one on no axiom; SHA-256 566615a6b9610fc981e03eb42e07af9e03849f20704729880f2bfe6434f2ea0b). For every actuation, the direction of its deed gives the fold,

the registration and the seat; the registration is the only height-keeping map onto the seat; the seat is the only place both blind spots cancel; no reading of the record returns the orientation; and every self-grounding root is the root of an actuation over its own deeds, so the root carries the three axes and is not characterless (actuation_is_tri axial, registration_is_forced, the_only_special_cut, tongue_freedom, every_root_actuates, the_root_carries_three_axes). The root's grade is read in the third kernel, R.3, Root_Grade_Ledger.lean (18 theorems, every one on no axiom; SHA-256 c18b68b29d2b6be5058d56d3a7f9e966bcde6ce099217e0f5945db4c829c42b8). On the grade ledger of the Master Codex (Islam 2026l), carried with its weakest-link join law, the Codex's rule reads a warranted root, one with a proof of its statement, at theorem grade, and an unwarranted root, the Codex's own declared axiom, at premise grade; the root of R.1 is warranted by a theorem with no axiom and no hypothesis, so on that rule it stands at theorem grade, the grade is read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA-Tongue co-location inherits the root's grade exactly (a_warranted_root_is_theorem_grade, the_ledger_reads_the_warrant, the_grade_is_not_elected, ra_is_theorem_grade, coloc_grade_is_the_roots_grade, the_ledger_seal). Both kernels compile with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0, declare no axiom, import nothing, and pin every cone as *does not depend on any axioms*; both are carried from the master plan of the series, Seven Rows, One Root (Islam 2026k), each extended in this revision of the series by one additive section, R.2 by its section IX and R.3 by its section VII, carried alike by every paper of the series, and the paper that reads them on its row cites them for exactly what they state.

```

/-
Triaxial_Actuation.lean · actuation is triaxial

```

Core Lean 4: no import, no axiom declared, no sorry, no native_decide, and every theorem rests on no axiom at all, not even propositional extensionality, quotients or choice.

- An actuation is a self-grounding root whose acts carry a direction: each act has a side and a height, and reversing the deed reverses the side and keeps the height. From the direction alone the kernel constructs three axes and proves them forced:
- I The fold: reversing the deed is an involution.
 - II The registration: it keeps the height and forgets the side; it is blind to the fold, closed with it under combination, and it is the only height-keeping map onto the seat.
 - III The seat: the fold's fixed set, where the fold and the registration both fix the act, and off which neither does.
 - IV The third axis: off the seat the fold's orbit has two acts with one record, so the record leaves exactly the orientation open; every reading blind to the fold reads both alike.
 - V The root through the axes: a reversed deed is still an act and instances the root; the axes are carried by every actuation, and they force no orientation.

```

VI The bridge atom: the integers, with negation as the fold, are
an actuation.
VII The triaxial seal.
VIII The Tongue: no reading of the record returns the orientation;
the seat is the only special cut.
IX Every root actuates: a self-grounding root is the root of an
actuation over its own acts, so the
root carries the three axes; the registration is the fold's
record and lands on the seat, where
the orientation an act and its reversal share stays open; the
even pair leaves two acts, the
orientation locks one; and a degenerate direction, one side
only, is the characterless case.
-/

namespace TriAct

/-! ## The root, self-grounding -/

/-- A self-grounding root: acts occur, and every act instances the
root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-! ## Actuation: a self-grounding root whose acts carry a direction
-/

/-- The direction of a deed: a side with a reversal, whose one fixed
side is the zero side. -/
structure Direction where
  Side      : Type
  rev       : Side → Side
  zero      : Side
  rev_rev   : ∀ s, rev (rev s) = s
  rev_zero  : rev zero = zero
  fix_zero  : ∀ s, rev s = s → s = zero

/-- An actuation: a self-grounding root whose acts are pairs of a
side and a height. -/
structure Actuation (R : Prop) where
  dir      : Direction
  Height   : Type
  anAct    : dir.Side × Height
  instances : dir.Side × Height → R

variable {R : Prop}

/-- Every actuation is a self-grounding root. -/
def Actuation.toSelfGrounding (A : Actuation R) : SelfGrounding R :=
  (A.dir.Side × A.Height, A.anAct, A.instances)

/-- The fold: reversing the deed reverses the side and keeps the
height. -/
def fold (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side
× A.Height := (A.dir.rev p.1, p.2)

/-- The registration: it keeps the height and forgets the side. -/
def reg (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side ×
A.Height := (A.dir.zero, p.2)

/-- The seat: the acts the fold fixes. -/
def OnSeat (A : Actuation R) (p : A.dir.Side × A.Height) : Prop :=
  fold A p = p

/-! ## I · the fold -/

theorem fold_involutive (A : Actuation R) (p : A.dir.Side ×
A.Height) : fold A (fold A p) = p := by
cases p with
| mk s h => show (A.dir.rev (A.dir.rev s), h) = (s, h); rw
[A.dir.rev_rev]

/-! ## II · the registration -/

/-- The registration forgets the side: an act and its reversal have
one record. -/
theorem reg_forgets_side (A : Actuation R) (p : A.dir.Side ×
A.Height) : reg A (fold A p) = reg A p := rfl

/-- The registration keeps the height. -/
theorem reg_keeps_height (A : Actuation R) (p : A.dir.Side ×
A.Height) : (reg A p).2 = p.2 := rfl

```

```

/-- The even pair is closed under combination: folding a record
changes nothing. -/
theorem fold_of_reg (A : Actuation R) (p : A.dir.Side × A.Height) :
fold A (reg A p) = reg A p := by
show (A.dir.rev A.dir.zero, p.2) = (A.dir.zero, p.2); rw
[A.dir.rev_zero]

/-- The registration is idempotent. -/
theorem reg_idempotent (A : Actuation R) (p : A.dir.Side × A.Height)
: reg A (reg A p) = reg A p := rfl

/-- THE REGISTRATION IS FORCED, NOT ELECTED: any map that keeps the
height and lands on the seat is
the registration. -/
theorem registration_is_forced (A : Actuation R) (g : A.dir.Side ×
A.Height → A.dir.Side × A.Height)
(keeps : ∀ p, (g p).2 = p.2) (lands : ∀ p, OnSeat A (g p)) : ∀ p,
g p = reg A p := by
intro p
have hs : (g p).1 = A.dir.zero := by
have hf := lands p
have hl : A.dir.rev (g p).1 = (g p).1 := congrArg Prod.fst hf
exact A.dir.fix_zero _ hl
have hh := keeps p
cases hg : g p with
| mk s h =>
rw [hg] at hs hh
show (s, h) = (A.dir.zero, p.2)
rw [show s = A.dir.zero from hs, show h = p.2 from hh]

/-! ## III · the seat -/

/-- The seat is exactly the zero side. -/
theorem seat_iff_zero_side (A : Actuation R) (p : A.dir.Side ×
A.Height) : OnSeat A p ↔ p.1 = A.dir.zero :=
(fun h => A.dir.fix_zero _ (congrArg Prod.fst h)),
fun h => by
cases p with
| mk s t =>
show (A.dir.rev s, t) = (s, t)
have hs : s = A.dir.zero := h
rw [hs, A.dir.rev_zero]

/-- ON THE SEAT BOTH BLIND SPOTS CANCEL: the fold and the
registration both fix the act. -/
theorem seat_cancels_both (A : Actuation R) (p : A.dir.Side ×
A.Height) (h : OnSeat A p) :
fold A p = p ∧ reg A p = p := by
refine (h, ?_)
have hz : p.1 = A.dir.zero := (seat_iff_zero_side A p).mp h
cases p with
| mk s t =>
have hs : s = A.dir.zero := hz
show (A.dir.zero, t) = (s, t)
rw [hs]

/-- OFF THE SEAT BOTH STAND: neither the fold nor the registration
fixes the act. -/
theorem off_seat_both (A : Actuation R) (p : A.dir.Side × A.Height)
(h : ¬ OnSeat A p) :
fold A p ≠ p ∧ reg A p ≠ p := by
refine (h, fun hr => h ?_)
have hz : p.1 = A.dir.zero := (congrArg Prod.fst hr).symm
exact (seat_iff_zero_side A p).mpr hz

/-! ## IV · the third axis -/

/-- THE RECORD LEAVES EXACTLY THE ORIENTATION: off the seat, an act
and its reversal are two acts with
one record. -/
theorem record_leaves_the_orientation (A : Actuation R) (p :
A.dir.Side × A.Height) (h : ¬ OnSeat A p) :
fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
(h, rfl)

/-- Every reading blind to the fold reads an act and its reversal
alike. -/
theorem even_readings_are_blind (A : Actuation R) {α : Type} (f :
A.dir.Side × A.Height → α)
(blind : ∀ p, f (fold A p) = f p) (p : A.dir.Side × A.Height) :
f (fold A p) = f p :=
blind p

```

```

/-- The registration is such a reading, and so is every reading of
the record. -/
theorem readings_of_the_record_are_blind (A : Actuation R) {α :
  Type} (k : A.dir.Side × A.Height → α)
  (p : A.dir.Side × A.Height) : k (reg A (fold A p)) = k (reg A p)
:= rfl

/-! ## V · the root through the axes -/

/-- A REVERSED DEED IS STILL AN ACT, AND INSTANCES THE ROOT: the
denial spends the deed. -/
theorem denial_instances_root (A : Actuation R) (p : A.dir.Side ×
  A.Height) : R :=
  A.instances (fold A p)

/-- The root holds on every actuation, by its act. -/
theorem root_holds (A : Actuation R) : R := A.instances A.act

/-- THE AXES FORCE NO ORIENTATION: off the seat both orientations
are acts, both instance the root, and
the record reads them alike; the triaxial structure leaves the one
orientation open. -/
theorem axes_force_no_orientation (A : Actuation R) (p : A.dir.Side
  × A.Height) (h : ¬ OnSeat A p) :
  R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
  (A.instances p, A.instances (fold A p), h, rfl)

/-! ## VI · the bridge atom: the integers are an actuation -/

theorem int_neg_neg : ∀ s : Int, -(s) = s
| Int.ofNat 0 => rfl
| Int.ofNat (n + 1) => rfl
| Int.negSucc _ => rfl

theorem self_neg_zero : ∀ v : Int, -v = v → v = 0
| Int.ofNat 0, _ => rfl
| Int.ofNat (n + 1), h => Int.noConfusion h
| Int.negSucc _, h => Int.noConfusion h

/-- The integer direction: negation, with zero its one fixed side. -/
def intDir : Direction := (Int, fun s => -s, 0, int_neg_neg, rfl,
  self_neg_zero)

/-- The bridge atom: points (d, t), the fold (d, t) ↔ (-d, t), the
registration (d, t) ↔ (0, t). -/
def atom : Actuation True := (intDir, Int, ((0 : Int), (0 : Int)),
  fun _ => True.intro)

/-- THE BRIDGE ATOM, AS AN ACTUATION: the fold fixes (d, t) exactly
when d = 0, the registration
leaves it unchanged exactly then, and off the seat the record leaves
the orientation. -/
theorem the_bridge_atom (d t : Int) :
  (OnSeat atom (d, t) ↔ d = 0) ∧ (reg atom (d, t) = (d, t) ↔ d = 0)
  ∧
  (d ≠ 0 → fold atom (d, t) ≠ (d, t) ∧ reg atom (fold atom (d, t))
    = reg atom (d, t)) :=
  (seat_iff_zero_side atom (d, t),
  (fun h => (congrArg Prod.fst h).symm, fun h => by rw [h]; rfl),
  fun hd => record_leaves_the_orientation atom (d, t) (fun hs => hd
    ((seat_iff_zero_side atom (d, t)).mp hs)))

/-- The atom is not characterless: off the seat it has two acts with
one record. -/
theorem the_atom_is_not_characterless :
  fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
  atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
    : Int)) :=
  (fun h => Int.noConfusion (congrArg Prod.fst h), rfl)

/-! ## VII · the triaxial seal -/

/-- ACTUATION IS TRIAXIAL: every actuation carries a fold that is an
involution, a registration that
keeps the height, forgets the side and is the only such map onto the
seat, and a seat where both fix
the act and off which neither does; off the seat the record leaves
exactly the orientation; every act
and its reversal instance the root; and the axes force no
orientation. -/
theorem actuation_is_triaxial (A : Actuation R) :
  (∀ p, fold A (fold A p) = p) ∧
  (∀ p, reg A (fold A p) = reg A p ∧ (reg A p).2 = p.2 ∧ fold A
    (reg A p) = reg A p) ∧

```

```

  (∀ g : A.dir.Side × A.Height → A.dir.Side × A.Height,
    (∀ p, (g p).2 = p.2) → (∀ p, OnSeat A (g p)) → ∀ p, g p = reg
    A p) ∧
  (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
  (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A p ≠ p) ∧
  (∀ p, ¬ OnSeat A p → R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) =
    reg A p) :=
  (fold_involutive A, fun p => (reg_forgets_side A p,
    reg_keeps_height A p, fold_of_reg A p),
  registration_is_forced A, seat_cancels_both A, off_seat_both A,
  axes_force_no_orientation A)

/-! THE TRIAXIAL SEAL: actuation is triaxial, universally; the
bridge atom is an actuation; and it is
not characterless. -/
theorem the_triaxial_seal :
  (∀ (R : Prop) (A : Actuation R), (∀ p, fold A (fold A p) = p) ∧
    (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
    (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A (fold A p) = reg A
      p)) ∧
  (∀ d t : Int, OnSeat atom (d, t) ↔ d = 0) ∧
  (fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
    atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
      : Int))) :=
  (fun _ => (fold_involutive A, seat_cancels_both A,
    fun p h => record_leaves_the_orientation A p h),
  fun d t => seat_iff_zero_side atom (d, t),
  the_atom_is_not_characterless)

/-! ## VIII · the Tongue: freedom and the only special cut -/

/-- TONGUE FREEDOM: no reading of the record returns the
orientation. If a record is blind to the fold
and an orientation is flipped by it, the orientation is not a
function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β →
  Bool) (d : α → Bool) (x : α)
  (blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g
  (ρ y) := by
  intro h
  have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
  have h2 : d x = g (ρ x) := congrFun h x
  rw [blind] at h1
  rw [← h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

/-! THE ONLY SPECIAL CUT: the seat is exactly where both blind spots
cancel; nowhere else do the fold
and the registration both fix an act. -/
theorem the_only_special_cut (A : Actuation R) (p : A.dir.Side ×
  A.Height) :
  (fold A p = p ∧ reg A p = p) ↔ OnSeat A p :=
  (fun h => h.1, fun h => seat_cancels_both A p h)

/-! ## IX · every root actuates: the root carries the three axes -/

/-- THE REGISTRATION IS THE FOLD'S RECORD: it reads an act and its
reversal alike and keeps the height, and two
acts have one record exactly when they have one height. -/
theorem registration_is_the_folds_record (A : Actuation R) (p q :
  A.dir.Side × A.Height) :
  reg A (fold A p) = reg A p ∧ (reg A p = reg A q ↔ p.2 = q.2) :=
  (rfl, (fun h => show (reg A p).2 = (reg A q).2 from congrArg
    Prod.snd h, fun h => by
    cases p with
    | mk s t =>
      cases q with
      | mk s' t' =>
        have ht : t = t' := h
        show (A.dir.zero, t) = (A.dir.zero, t')
        rw [ht]))

/-! THE REGISTRATION LANDS ON THE SEAT: every record is a seat
point. -/
theorem reg_lands_on_seat (A : Actuation R) (p : A.dir.Side ×
  A.Height) : OnSeat A (reg A p) :=
  fold_of_reg A p

/-! THE ORIENTATION IS OPEN AT THE SEAT: off the seat, an act and
its reversal land on one seat point, the record

```

```

of both; at that place the record has left exactly the orientation
open. -/
theorem open_at_the_seat (A : Actuation R) (p : A.dir.Side ×
  A.Height) (h : ¬ OnSeat A p) :
  OnSeat A (reg A p) ∧ reg A (fold A p) = reg A p ∧ fold A p ≠ p :=
  (reg_lands_on_seat A p, rfl, h)

/-- TWO AXES LEAVE TWO: off the seat, every reading blind to the
fold, the record among them, reads an act and
its reversal alike: two acts, one reading. -/
theorem even_pair_leaves_two (A : Actuation R) (p : A.dir.Side ×
  A.Height) (h : ¬ OnSeat A p) {α : Type}
  (f : A.dir.Side × A.Height → α) (blind : ∀ q, f (fold A q) = f
  q) :
  fold A p ≠ p ∧ f (fold A p) = f p ∧ reg A (fold A p) = reg A p :=
  (h, blind p, rfl)

/-- THREE AXES LOCK ONE: the record together with the orientation,
the side, determines the act. -/
theorem orientation_locks_the_act (A : Actuation R) (p q :
  A.dir.Side × A.Height)
  (hr : reg A p = reg A q) (hs : p.1 = q.1) : p = q := by
  cases p with
  | mk s t =>
    cases q with
    | mk s' t' =>
      have ht : t = t' := congrArg Prod.snd hr
      have hs' : s = s' := hs
      rw [hs', ht]

/-- A degenerate direction has the zero side and no other. -/
def Direction.Degenerate (D : Direction) : Prop := ∀ s, s = D.zero

/-- THE CHARACTERLESS CASE: over a degenerate direction every act is
on the seat, and the fold and the
registration are both the identity: one scalar, no fold, no seat to
lock. -/
theorem degenerate_is_characterless (A : Actuation R) (h :
  A.dir.Degenerate) (p : A.dir.Side × A.Height) :
  OnSeat A p ∧ fold A p = p ∧ reg A p = p :=
  have hs : OnSeat A p := (seat_iff_zero_side A p).mpr (h p.1)
  (hs, seat_cancels_both A p hs)

/-- The integer direction is not degenerate. -/
theorem intDir_not_degenerate : ¬ intDir.Degenerate := fun h =>
  have h0 : (1 : Int) = 0 := h (1 : Int)
  absurd h0 (by decide)

/-- EVERY ROOT ACTUATES: a self-grounding root is the root of the
actuation over its own acts, with the integer
direction; the root's deeds sit at the zero side and instance the
root as before. -/
def SelfGrounding.toActuation (S : SelfGrounding R) : Actuation R :=
  (intDir, S.Act, ((0 : Int), S.anAct), fun p => S.instances p.2)

theorem every_root_actuates (S : SelfGrounding R) :
  (∀ a : S.Act, OnSeat S.toActuation ((0 : Int), a)) ∧
  (∀ a : S.Act, ¬ OnSeat S.toActuation ((1 : Int), a)) ∧
  ¬ S.toActuation.dir.Degenerate :=
  (fun _ => rfl,
  fun a h =>
    have h0 : (1 : Int) = 0 := (seat_iff_zero_side S.toActuation
      ((1 : Int), a)).mp h
    absurd h0 (by decide),
  intDir_not_degenerate)

/-- THE ROOT CARRIES THREE AXES: for every self-grounding root, its
actuation carries the fold, an involution; the
registration, keeping the height, forgetting the side, the only
height-keeping map onto the seat; and the seat,
where both fix the act; and it is not characterless: at every height
an act off the seat, its reversal a second
act with the same record, both instantiating the root, the orientation
between them forced by nothing. -/
theorem the_root_carries_three_axes (S : SelfGrounding R) :
  (∀ p, fold S.toActuation (fold S.toActuation p) = p) ∧
  (∀ p, reg S.toActuation (fold S.toActuation p) = reg
  S.toActuation p ∧ (reg S.toActuation p).2 = p.2) ∧
  (∀ g : Int × S.Act → Int × S.Act, (∀ p, (g p).2 = p.2) → (∀ p,
  OnSeat S.toActuation (g p) →
    ∀ p, g p = reg S.toActuation p) ∧
  (∀ p, OnSeat S.toActuation p → fold S.toActuation p = p) ∧ reg
  S.toActuation p = p) ∧
  (∀ a : S.Act, fold S.toActuation ((1 : Int), a) ≠ ((1 : Int), a) ∧

```

```

  reg S.toActuation (fold S.toActuation ((1 : Int), a)) = reg
  S.toActuation ((1 : Int), a)) ∧
  (∀ p, ¬ OnSeat S.toActuation p →
    R A R ∧ fold S.toActuation p ≠ p ∧ reg S.toActuation (fold
  S.toActuation p) = reg S.toActuation p) :=
  (fold_involutive S.toActuation,
  fun p => (reg_forgets_side S.toActuation p, reg_keeps_height
  S.toActuation p),
  registration_is_forced S.toActuation,
  seat_cancels_both S.toActuation,
  fun a => record_leaves_the_orientation S.toActuation ((1 : Int),
  a) ((every_root_actuates S).2.1 a),
  axes_force_no_orientation S.toActuation)

end TriAct

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'TriAct.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_involutive
/-- info: 'TriAct.reg_forgets_side' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_forgets_side
/-- info: 'TriAct.reg_keeps_height' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_keeps_height
/-- info: 'TriAct.fold_of_reg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_of_reg
/-- info: 'TriAct.reg_idempotent' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_idempotent
/-- info: 'TriAct.registration_is_forced' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.registration_is_forced
/-- info: 'TriAct.seat_iff_zero_side' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.seat_iff_zero_side
/-- info: 'TriAct.seat_cancels_both' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.seat_cancels_both
/-- info: 'TriAct.off_seat_both' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.off_seat_both
/-- info: 'TriAct.record_leaves_the_orientation' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.record_leaves_the_orientation
/-- info: 'TriAct.even_readings_are_blind' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_readings_are_blind
/-- info: 'TriAct.readings_of_the_record_are_blind' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.readings_of_the_record_are_blind
/-- info: 'TriAct.denial_instances_root' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.denial_instances_root
/-- info: 'TriAct.root_holds' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.root_holds
/-- info: 'TriAct.axes_force_no_orientation' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.axes_force_no_orientation
/-- info: 'TriAct.int_neg_neg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.int_neg_neg
/-- info: 'TriAct.self_neg_zero' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.self_neg_zero
/-- info: 'TriAct.the_bridge_atom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_bridge_atom
/-- info: 'TriAct.the_atom_is_not_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_atom_is_not_characterless
/-- info: 'TriAct.actuation_is_tri axial' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.actuation_is_tri axial
/-- info: 'TriAct.the_tri axial_seal' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.the_tri axial_seal
/-- info: 'TriAct.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.tongue_freedom
/-- info: 'TriAct.the_only_special_cut' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.the_only_special_cut
/-- info: 'TriAct.registration_is_the_folds_record' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.registration_is_the_folds_record
/-- info: 'TriAct.reg_lands_on_seat' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.reg_lands_on_seat
/-- info: 'TriAct.open_at_the_seat' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.open_at_the_seat

```

```

/-- info: 'TriAct.even_pair_leaves_two' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_pair_leaves_two
/-- info: 'TriAct.orientation_locks_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.orientation_locks_the_act
/-- info: 'TriAct.degenerate_is_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.degenerate_is_characterless
/-- info: 'TriAct.intDir_not_degenerate' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.intDir_not_degenerate
/-- info: 'TriAct.every_root_actuates' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.every_root_actuates
/-- info: 'TriAct.the_root_carries_three_axes' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_root_carries_three_axes

/-
Root_Grade_Ledger.lean · the root at theorem grade, and RA-Tongue
co-location with it

Core Lean 4: no import, no axiom declared, no sorry, no
native_decide, and every theorem rests on no
axiom at all, not even propositional extensionality, quotients or
choice.

The floor Codex (Master Codex v5.1.0, Codex.lean) grades every
claim on one ledger and joins claims
at the weakest link. It declares the root as `axiom RA` and pins
`raClaim.grade = .premise`; its
co-location claim, the root joined with the wall, therefore
inherits `.premise`. This kernel carries
the same ledger and the same join law with no axiom declared,
warrants the root by a theorem with no
axiom and no hypothesis, and proves that the co-location claim
then stands at theorem grade.
I The grade ledger and the weakest-link join, as in the Codex.
II The root, a theorem: the constructed domain of TOE_Zero.lean,
warranted on no axiom.
III The wall, a theorem: the Codex's K4 frame, its target not a
function of its record, proved on
no axiom through Tongue freedom.
IV The claims: the root at theorem grade, the wall at theorem
grade, and their co-location.
V Why the Codex read premise: a join's grade is its weakest
link, and co-location's grade is the
root's grade exactly; with the root at theorem grade,
co-location is a theorem.
VI The ledger seal.
VII The grade read from the warrant, as the Codex reads it: a
root with a warrant, a proof of its
statement, is theorem grade; a root posited without one, the
Codex's `axiom RA`, is premise grade;
the two claims of section IV carry exactly the grades this
rule reads.
-/

namespace RGL

/-! ## I · the grade ledger, as in the Codex -/

inductive Grade : Type
| premise | corroboration | operational | structural | engineering
| conditional | theoremConditional | analytic | theorem
deriving DecidableEq

def Grade.rank : Grade → Nat
| .premise => 0 | .corroboration => 1 | .operational => 2 |
|.structural => 3
| .engineering => 4 | .conditional => 5 | .theoremConditional => 6
| .analytic => 7 | .theorem => 8

def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a
else b

/-- A claim carries its statement, its grade and its warrant, a
proof of the statement. -/
structure Claim : Type where
stmt : Prop
grade : Grade
warrant : stmt

/-- Two claims join at their weakest link. -/

```

```

def Claim.join (c₁ c₂ : Claim) : Claim where
stmt := c₁.stmt ∧ c₂.stmt
grade := Grade.weakest c₁.grade c₂.grade
warrant := (c₁.warrant, c₂.warrant)

theorem join_grade (c₁ c₂ : Claim) : (Claim.join c₁ c₂).grade =
Grade.weakest c₁.grade c₂.grade := rfl

/-! ## II · the root, a theorem (the constructed domain of
TOE_Zero.lean) -/

structure SelfGrounding (R : Prop) where
Act : Type
anAct : Act
instances : Act → R

theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
G.instances G.anAct

def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ => show (0 : Int)
< 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM: no axiom, no
hypothesis. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
ra₀

/-! ## III · the wall, a theorem (the Codex's K4 frame) -/

/-- Tongue freedom: if a record is blind to the fold and an
orientation is flipped by it, the
orientation is not a function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β →
Bool) (d : α → Bool) (x : α)
(blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g
(ρ y) := by
intro h
have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
have h2 : d x = g (ρ x) := congrFun h x
rw [blind] at h1
rw [h2] at h1
rw [h1] at flip
cases hd : d x with
| false => rw [hd] at flip; exact Bool.noConfusion flip
| true => rw [hd] at flip; exact Bool.noConfusion flip

namespace K4

def val : Nat → Nat
| 0 => 11 | 1 => 851 | 2 => 13 | 3 => 1273 | 4 => 17 | 5 => 437
| 6 => 19 | 7 => 2119 | 8 => 23 | 9 => 1703 | 10 => 29 | 11 => 869
| _ => 0

def tau : Nat → Nat := fun i =>
if i < 12 then (if i % 2 = 0 then i + 1 else i - 1) else i

def target : Nat → Bool := fun i => i % 2 == 0

def rho : Nat → Nat := fun i =>
(val i % 2) * 1000 + (val i % 3) * 100 + (val i % 5) * 10 + (val i
% 7)

/-- The record is blind to the fold at the first pair. -/
theorem register_even_at_zero : rho (tau 0) = rho 0 := by decide

/-- The target is flipped by the fold at the first pair. -/
theorem target_odd_at_zero : target (tau 0) = !target 0 := by decide

/-- THE WALL: no reading of the record returns the target. -/
theorem wall (g : Nat → Bool) : target ≠ fun y => g (rho y) :=
tongue_freedom tau rho g target 0 register_even_at_zero
target_odd_at_zero

end K4

/-! ## IV · the claims -/

/-- The root at theorem grade, warranted by a theorem with no axiom
and no hypothesis. -/
def raClaim₀ : Claim where
stmt := RA₀
grade := .theorem

```

```

warrant := root_on_the_constructed_domain

/-- The wall at theorem grade, as in the Codex. -/
def wallClaim : Claim where
  stmt :=  $\forall g : \text{Nat} \rightarrow \text{Bool}, K4.\text{target} \neq \text{fun } y \Rightarrow g (K4.\text{rho } y)$ 
  grade := .theorem
  warrant := K4.wall

/-- RA-Tongue co-location: the root joined with the wall, as in the Codex. -/
def colocClaim0 : Claim := Claim.join raClaim0 wallClaim

/-- The Codex's root, graded as the Codex grades it, for comparison: the same statement form, at premise grade. -/
def raClaimPremise : Claim where
  stmt := RA0
  grade := .premise
  warrant := root_on_the_constructed_domain

/-! ## V · why the Codex read premise, and what co-location is now -/

/-- A join with a theorem-grade claim has exactly the other claim's grade. -/
theorem weakest_with_theorem (g : Grade) : Grade.weakest g .theorem = g := by
  cases g <|> rfl

/-- CO-LOCATION'S GRADE IS THE ROOT'S GRADE, EXACTLY: the wall is a theorem, so the join reads the root's grade and nothing else. -/
theorem coloc_grade_is_the_roots_grade (r : Claim) :
  (Claim.join r wallClaim).grade = r.grade := by
  show Grade.weakest r.grade Grade.theorem = r.grade
  exact weakest_with_theorem r.grade

/-- WHY THE CODEX READ PREMISE: with the root labelled premise, the co-location claim is premise. -/
theorem premise_root_gives_premise_coloc : (Claim.join raClaimPremise wallClaim).grade = .premise := rfl

/-- THE ROOT IS A THEOREM-GRADE CLAIM. -/
theorem ra_is_theorem_grade : raClaim0.grade = .theorem := rfl

/-- RA-TONGUE CO-LOCATION IS A THEOREM-GRADE CLAIM. -/
theorem coloc_is_theorem_grade : colocClaim0.grade = .theorem := rfl

/-- The co-location claim's statement holds: the root and the wall, both proved. -/
theorem coloc_holds : colocClaim0.stmt := colocClaim0.warrant

/-! ## VI · the ledger seal -/

/-- THE LEDGER SEAL: the root is a theorem on no axiom and no hypothesis; the wall is a theorem; their co-location holds and stands at theorem grade; and co-location's grade is the root's grade exactly, which is why the Codex, labelling the root premise, read premise. -/
theorem the_ledger_seal :
  RA0  $\wedge$  ( $\forall g : \text{Nat} \rightarrow \text{Bool}, K4.\text{target} \neq \text{fun } y \Rightarrow g (K4.\text{rho } y)$ )  $\wedge$ 
  colocClaim0.stmt  $\wedge$  colocClaim0.grade = .theorem  $\wedge$ 
  ( $\forall r : \text{Claim}, (\text{Claim.join } r \text{ wallClaim}).\text{grade} = r.\text{grade}$ )  $\wedge$ 
  (Claim.join raClaimPremise wallClaim).grade = .premise :=
  (root_on_the_constructed_domain, K4.wall, coloc_holds,
   coloc_is_theorem_grade,
   coloc_grade_is_the_roots_grade, premise_root_gives_premise_coloc)

/-! ## VII · the grade read from the warrant, as the Codex reads it -/

/-- The evidence a ledger entry carries for a statement: posited with no warrant, or warranted by a proof. -/
inductive Evidence (P : Prop) : Type
| posited
| warranted (w : P)

/-- THE CODEX'S RULE FOR THE ROOT'S GRADE: evidence at theorem grade is a proof with its axioms printed. A root that comes with a warrant, a proof of its statement, is theorem grade; a root posited with none, as the Codex declares 'axiom RA', is premise grade. -/
def rootGrade : Evidence RA0 → Grade
| .warranted _ => .theorem
| .posited => .premise

```

```

/-- A WARRANTED ROOT IS THEOREM GRADE: the root of section II carries its warrant, so the rule reads theorem. -/
theorem a_warranted_root_is_theorem_grade :
  rootGrade (.warranted root_on_the_constructed_domain) = .theorem := rfl

/-- AN UNWARRANTED ROOT IS PREMISE GRADE: with no warrant the rule reads premise, the Codex's own reading of its declared axiom. -/
theorem an_unwarranted_root_is_premise_grade : rootGrade .posited = .premise := rfl

/-- THE LEDGER READS THE WARRANT: the root claim of section IV carries the grade the rule reads from its warrant, and the comparison claim carries the grade the rule reads from none. -/
theorem the_ledger_reads_the_warrant :
  raClaim0.grade = rootGrade (.warranted raClaim0.warrant)  $\wedge$ 
  raClaimPremise.grade = rootGrade .posited :=
  (rfl, rfl)

/-- THE GRADE IS NOT ELECTED: the rule reads theorem from a warrant and premise from none, never the other way; whatever label an entry carries, the warranted root's grade under the rule is theorem. -/
theorem the_grade_is_not_elected :
  ( $\forall w : \text{RA}_0, \text{rootGrade } (.warranted w) = .\text{theorem}$ )  $\wedge$ 
  ( $\forall w : \text{RA}_0, \text{rootGrade } (.warranted w) \neq \text{rootGrade } .\text{posited}$ ) :=
  (fun _ => rfl, fun h => Grade.noConfusion h, fun _ h => Grade.noConfusion h)

end RGL

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-! info: 'RGL.join_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.join_grade
/-! info: 'RGL.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.seated_undeniable
/-! info: 'RGL.root_on_the_constructed_domain' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.root_on_the_constructed_domain
/-! info: 'RGL.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.tongue_freedom
/-! info: 'RGL.K4.register_even_at_zero' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.register_even_at_zero
/-! info: 'RGL.K4.target_odd_at_zero' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.target_odd_at_zero
/-! info: 'RGL.K4.wall' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.wall
/-! info: 'RGL.weakest_with_theorem' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.weakest_with_theorem
/-! info: 'RGL.coloc_grade_is_the_roots_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_grade_is_the_roots_grade
/-! info: 'RGL.premise_root_gives_premise_coloc' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.premise_root_gives_premise_coloc
/-! info: 'RGL.ra_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.ra_is_theorem_grade
/-! info: 'RGL.coloc_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_is_theorem_grade
/-! info: 'RGL.coloc_holds' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_holds
/-! info: 'RGL.the_ledger_seal' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_ledger_seal
/-! info: 'RGL.a_warranted_root_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.a_warranted_root_is_theorem_grade
/-! info: 'RGL.an_unwarranted_root_is_premise_grade' does not depend on any axioms -/
#guard_msgs in #print axioms
  RGL.an_unwarranted_root_is_premise_grade
/-! info: 'RGL.the_ledger_reads_the_warrant' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_ledger_reads_the_warrant
/-! info: 'RGL.the_grade_is_not_elected' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_grade_is_not_elected

```

AUTHOR'S PROVENANCE AND METHOD DISCLOSURE

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full at the reference below.

Author and method. Mohammad F. Islam, PhD, independent researcher, sole author and sole authority for every claim and every error. The reduction was directed by the author and drafted with Claude (Anthropic) as scribe: the model wrote and compiled the kernel, checked citations and argued against the author's positions on instruction; it holds no authorship. One claim was held at the model's insistence: the paper does not state that regularity follows from existence as given, because the kernel's own `given_is_not_the_value` proves the contrary.

The register. In the programme's vocabulary: the form, the act's identity with the value, the universal closure and the kinetic theorem stand at $[\{\epsilon\}, T]$ on no axiom; the triaxial reading at $[\{\epsilon\}, S]$; the value at the continuum at premise grade on the act; $\Delta M = 0$ on the cited physics. The act is the row's least-erasure posit read as existence acting finitely; the row carries no defeater. The freedom cut, the triaxial cut of Section 5 on the row's bit, prime-as-freedom and the triaxial lock are applied on the row. The effective alignment-defect premise of the earlier papers is retired as a placeholder.

Audit. This edition has not yet been prosecuted by the external substrates, and no audit result is claimed. The development provenance is the author's Zenodo trail, the Unabridged Trisduction Codex and the Lean Codex at github.com/1000sapient/Trisduction.

Nature of Root Axiom

The root is triaxial, and that is the force of the cut. A characterless existence has no triaxiality: one scalar, no fold, no seat, nothing to lock. The root, to exist is to actuate, carries three axes, and they are not chosen: for every actuation, the direction of its deed gives the fold, the registration and the seat, and the registration is the only height-keeping map onto the seat. The deed is the direction: a denial spends the deed and instances the root. The registration is the fold's record: it keeps the height and forgets the side, and the two even axes, symmetry and record, are closed under combination and blind. The seat is the fixed set, where the two blind spots cancel, and the third axis is the one orientation the even pair cannot supply. Two axes never determine the point; three do. The seven rows are one cut because each is an involution with a seat and one missing orientation. The root makes the third axis native as a place: the seat, and the one orientation still open there; the axes themselves force no orientation. Which way that orientation points is the value, and it is the root read on that seat. The root alone still forces no value.

Key to the statement, in the words of its receipts. The three axes are the two even axes, the fold's symmetry and the record, and the orientation, the side of a point of the actuation, which the even pair cannot supply; the fold, the registration and the seat are what the direction of the deed gives, and the seat is where the side is read. The point two

axes never determine is a point off the seat: there the even pair does not tell a point from its reversal, and the record with the side locks the point; on the seat both blind spots cancel and nothing is open there to determine. A point of the actuation is a side and a height, and every point instances the root: the points are the actuation's deeds, and the deed is the direction in the sense that its side is its direction, reversed by the fold. The characterless case, one scalar, is a degenerate direction: the fold is the identity and the seat is total, which is to say no fold and no seat to lock. The axes are not chosen in the sense that, given the direction, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat; the root's own actuation takes the integer direction, under which it is not characterless, while a degenerate direction over the same deeds would be (`every_root_actuates`, `degenerate_is_characterless`). The root read on that seat is the root read on the row at the seat point where a point and its reversal land with one record; that the orientation between them is the row's value bit is the paper's structural reading, a second statement beside its kernel's theorems, and no kernel states a map between the two. Each row carries the actuation's seat as the root of an actuation over its own deeds; a row's own seat, where its kernel states one, is a further statement of that kernel, named in the last row of the table, and on P versus NP it is vacant. A deed of the root is an inhabitant of its type of acts, which the root kernel's own comments call an act; in the root's actuation those deeds sit at the zero side, on the seat, and the points off the seat are the sides the direction supplies; the act, unqualified, is existence read on the row, the field the closing theorem consumes.

Receipts, each cited for what its statement says; R.1, R.2 and R.3 are the three kernels of Appendix R, every theorem of each on no axiom; the three theorems of the Master Codex are cited, not carried.

Clause of the statement	Receipt
the root carries three axes, not chosen: every self-grounding root is the root of an actuation over its own deeds, and for every actuation, given the direction of its deed, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat	<code>every_root_actuates</code> , <code>the_root_carries_three_axes</code> , <code>actuation_is_triaxial</code> , <code>registration_is_forced</code> (Appendix R.2)
the registration is the only height-keeping map onto the seat	<code>registration_is_forced</code> (R.2)
the axes force no orientation	<code>axes_force_no_orientation</code> (R.2)
the deed is the direction: a deed of the actuation is a point, a side and a height, its side its direction, reversed by the fold; a reversed point is still a deed and instances the root; a denial spends the deed and instances the root	<code>fold</code> , <code>denial_instances_root</code> (R.2); <code>denial_reenacts_root</code> , <code>denial_instantiates</code> (R.1)
the registration is the fold's record: it keeps the height, forgets the side and reads a point and its reversal alike; the fold is an involution; the even pair is closed under combination	<code>registration_is_the_folds_record</code> , <code>reg_forgets_side</code> , <code>reg_keeps_height</code> , <code>fold_involution</code> , <code>fold_of_reg</code> (R.2)

Clause of the statement	Receipt
the two even axes are blind: no reading of the record returns the orientation; symmetry keyless, the line property keyed	readings_of_the_record_are_blind, tongue_freedom (R.2); symmetry_is_keyless, line_property_is_keyed, exactly_one_stands (Islam 2026l, Master Codex v5.1.0, commit 5e55fca, cited for their statements, not carried; that Codex declares its root as an axiom, this paper's kernels declare none)
the seat is the fixed set, where both blind spots cancel; off the seat both stand	seat_iff_zero_side, seat_cancels_both, off_seat_both, the_only_special_cut (R.2)
the registration lands on the seat; off the seat a point and its reversal land on one seat point with one record, and the orientation is what stays open there; every reading of the record reads both alike	reg_lands_on_seat, open_at_the_seat, record_leaves_the_orientation, readings_of_the_record_are_blind (R.2)
the characterless case, no fold and no seat to lock: over a degenerate direction, one side only, the fold is the identity and every point is on the seat; the root's actuation is not degenerate	degenerate_is_characterless, intDir_not_degenerate, every_root_actuates (R.2)
the bridge atom is an actuation, and not characterless	the_bridge_atom, the_atom_is_not_characterless, the_triaxial_seal (R.2)
the scalar ground in the algebra, a third reading beside the actuation's: the Return, the line the conjugation fixes and the twelve gates, bound with the root	the_return, the_line_is_the_fixed_set, twenty_four_units_twelve_gates, the_class_equation, the_master_seal (R.1)
the root at theorem grade on the Codex's ledger rule, the grade read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA--Tongue co-location inheriting it	a_warranted_root_is_theorem_grade, the_ledger_reads_the_warrant, the_grade_is_not_elected, ra_is_theorem_grade, coloc_grade_is_the_roots_grade, the_ledger_seal (R.3)
two axes never determine the point; three do: off the seat the even pair does not tell a point from its reversal, and the record with the side, which carries the orientation, locks the point; on the seat both blind spots cancel and nothing is open to determine	even_pair_leaves_two, orientation_locks_the_act, seat_cancels_both (R.2); in its own coordinates, for independent axes in $\mathbb{F}_2^3$, two_axes_leave_two, three_axes_lock_one (this paper's kernel), a second statement

Clause of the statement	Receipt
the root alone forces no value; the root read on the row is the value	undeniable_root_forces_no_value; root_on_row_is_the_value, act_is_the_value (this paper's kernel)
the seven rows are one cut: each row's root is an actuation, so each carries an involution, the fold, with a seat and one missing orientation	every_root_actuates, the_root_carries_three_axes (R.2), stated for every self-grounding root and read at the constructed root <code>ra</code> of this paper's kernel, whose structure repeats R.2's SelfGrounding field for field; this kernel states no seat of its own; its fold on the value bit is the swap of the two worlds that share one record, the Bool model (<code>record_wall</code> , <code>fibre_is_two</code> , <code>one_free_orbit</code>), and the seat its actuation carries is the record both deeds land on; the row ledger of Islam (2026h, Book III), cited

REFERENCES

- de Moura, L. and S. Ullrich. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE 28*, 625--635.
- Fefferman, C. 2006. Existence and smoothness of the Navier--Stokes equation. In *The Millennium Prize Problems*, 57--67. Clay Mathematics Institute.
- Fujita, H. and T. Kato. 1964. On the Navier--Stokes initial value problem. I. *Archive for Rational Mechanics and Analysis* 16: 269--315.
- Islam, M. F. 2026a. Integrable misalignment forbids blowup. Zenodo. doi:10.5281/zenodo.22665832.
- Islam, M. F. 2026b. Global smoothness of the three-dimensional Navier--Stokes equations from a single effective axiom. Zenodo. doi:10.5281/zenodo.22670344.
- Islam, M. F. 2026c. The forced alternative is a filter. Zenodo. doi:10.5281/zenodo.22670253.
- Islam, M. F. 2026d. Closure and the limits of forced results. Zenodo. doi:10.5281/zenodo.22683806.
- Islam, M. F. 2026e. A formal proof of Navier--Stokes termination at the formal-alone register. Zenodo. doi:10.5281/zenodo.22705897.
- Islam, M. F. 2026f. The fluid is the witness. Zenodo. doi:10.5281/zenodo.22986563.
- Islam, M. F. 2026g. A formal proof of the Riemann Hypothesis by least erasure. Zenodo. doi:10.5281/zenodo.23034066.
- Islam, M. F. 2026h. Nothing escapes least erasure. Zenodo. doi:10.5281/zenodo.23080221.
- Islam, M. F. 2026i. PhysOS^T: the Trisduction physical operating system. Zenodo. doi:10.5281/zenodo.23117440.
- Islam, M. F. 2026j. Theory of Theories of Everything (TOE of All TOEs): existence proves existence only by motion. Zenodo. doi:10.5281/zenodo.23168379.
- Islam, M. F. 2026k. Seven rows, one root: the master plan of the series, revision F.5. Manuscript of record, 5 October 2026,

- carried with the series; its Appendix R kernels are carried into this paper's Appendix R, each extended by one additive section.
- Islam, M. F. 2026l. Trisduction: the Master Codex, edition 5.1.0. Codex.lean and its kernels, commit 5e55fca98c2f, master SHA-256 441a485d7c3bd3ce. Repository 1000sapients/Trisduction, GitHub.
- Islam, M. TRISDUCTION: A linguistically, topologically, and mathematically sealed verification architecture. Triaxial orthogonality, twelve-gate closure, the quaternionic completion, the root axiom, and the master pre-sealed proposition ledger. Zenodo, version 4, 19 June 2026. doi:10.5281/zenodo.20757507. Mirror: PhilArchive record ISLTTG. Master reference, continuously updated at the same location.
- Leray, J. 1934. Sur le mouvement d'un liquide visqueux emplissant l'espace. *Acta Mathematica* 63: 193--248.
- Mandelstam, L. and I. Tamm. 1945. The uncertainty relation between energy and time in non-relativistic quantum mechanics. *Journal of Physics (USSR)* 9: 249--254.
- Margolus, N. and L. B. Levitin. 1998. The maximum speed of dynamical evolution. *Physica D* 120: 188--195.
- Tao, T. 2016. Finite time blowup for an averaged three-dimensional Navier--Stokes equation. *Journal of the American Mathematical Society* 29: 601--674.

End of manuscript